

HỌC VIỆN KỸ THUẬT MẬT MÃ

KHOA ĐIỆN TỬ - VI MẠCH



BÁO CÁO MÔN HỌC

THIẾT KẾ HỆ THỐNG NHÚNG

IP ĐỌC GIÁ TRỊ CẢM BIẾN THEO CHU KỲ

Giảng viên hướng dẫn: TH.S Nguyễn Thanh Ngọc

Sinh viên thực hiện: Hà Văn Đạt – DT060209

Hà Nội – 2025

LỜI NÓI ĐẦU

Trong bối cảnh công nghệ bán dẫn và hệ thống nhúng không ngừng phát triển, lĩnh vực điện tử vi mạch ngày càng giữ vai trò quan trọng trong việc thiết kế và triển khai các hệ thống xử lý tín hiệu có độ chính xác cao, hoạt động ổn định và linh hoạt. Đặc biệt, việc xây dựng các IP core chuyên dụng cho từng chức năng cụ thể giúp tối ưu hóa tài nguyên phần cứng, đồng thời nâng cao khả năng tái sử dụng và mở rộng của hệ thống.

Xuất phát từ nhu cầu đó, đề tài “Thiết kế IP đọc giá trị cảm biến theo chu kỳ” được thực hiện nhằm nghiên cứu và xây dựng một hệ thống IP hoàn chỉnh, cho phép thu thập dữ liệu cảm biến một cách tự động và tuần hoàn theo thời gian định sẵn. Đề tài được thực hiện dưới sự hướng dẫn của Thạc sĩ Nguyễn Thanh Ngọc, với yêu cầu thiết kế hai IP chính gồm IP đọc dữ liệu cảm biến và IP Timer dùng để điều khiển chu kỳ lấy mẫu của hệ thống.

Trong phạm vi nghiên cứu, hai loại cảm biến được lựa chọn là cảm biến nhiệt độ – độ ẩm DHT11 và cảm biến ánh sáng BH1750. Cảm biến DHT11 cho phép đo và giám sát các thông số nhiệt độ và độ ẩm môi trường, trong khi cảm biến BH1750 cung cấp giá trị cường độ ánh sáng ở dạng số thông qua giao tiếp I²C, phù hợp cho các ứng dụng đo lường môi trường. Việc kết hợp hai loại cảm biến này giúp hệ thống thể hiện khả năng thu thập và xử lý dữ liệu từ cả hai dạng tín hiệu khác nhau, đồng thời đáp ứng tốt mục tiêu nghiên cứu và học tập trong lĩnh vực điện tử vi mạch.

Thông qua đề tài, em mong muốn củng cố kiến thức về thiết kế IP phần cứng, giao tiếp cảm biến và quy trình xây dựng, kiểm thử hệ thống số. Đồng thời, kết quả đạt được từ đề tài có thể làm cơ sở cho việc phát triển và mở rộng các ứng dụng trong các hệ thống đo lường, giám sát và điều khiển thông minh trong tương lai.

Em xin chân thành cảm ơn!

MỤC LỤC

LỜI NÓI ĐẦU	i
Danh sách hình ảnh	iii
Danh sách bảng	iv
CƠ SỞ LÝ THUYẾT	1
1.1 Mục tiêu đề tài	1
1.2 Cơ sở kỹ thuật và linh kiện	1
1.2.1 Nền tảng phần cứng: FPGA Arty A7-35T	1
1.2.2 Module cảm biến ánh sáng BH1750	3
1.2.3 Cảm biến nhiệt độ và độ ẩm DHT11	4
1.2.4 Timer tích hợp trên FPGA	6
1.3 Giới thiệu về giao tiếp AXI	7
1.3.1 Tổng quan AXI và AMBA	7
1.3.2 AXI4-Lite và lý do lựa chọn	8
1.3.3 Cơ chế bắt tay VALID/READY	9
1.3.4 Quá trình đọc/ghi trong AXI4-Lite	11
1.3.5 Ánh xạ thanh ghi AXI4-Lite cho IP đọc cảm biến	13
1.4 Công nghệ phần mềm	13
1.4.1 Vivado	13
1.4.2 Vitis	14
THIẾT IP ĐỌC CẢM BIẾN	16
2.1 Tổng quan kiến trúc hệ thống	16
2.2 Thiết kế IP đọc cảm biến	16
2.2.1 Thiết kế ip BH1750	16
2.2.2 Thiết kế IP DHT11	26
Kết quả thực nghiệm và đánh giá	35
3.1 Xây dựng hệ thống trên Vivado	35
3.2 Kết quả và đánh giá	38
Kết luận và hướng phát triển	40
4.1 Ưu điểm	40
4.2 Nhược điểm	40
4.3 Hướng phát triển	41

DANH SÁCH HÌNH ẢNH

1	Bo mạch phát triển Arty A7-35T	1
2	Module cảm biến ánh sáng BH1750	3
3	Sơ đồ cấu hình chân BH1750	4
4	DHT11 thực tế	4
5	Sơ đồ khối AXI Timer/Counter IP Core	6
6	Kịch bản VALID được khẳng định trước READY	10
7	Kịch bản READY được khẳng định trước VALID	10
8	Kịch bản VALID và READY cùng được khẳng định trong một chu kỳ clock	10
9	Channel architecture of writes	11
10	Channel architecture of writes	12
11	Lưu đồ Measurement Procedure của cảm biến BH1750	17
12	Trình tự thời gian đo trong chế độ Continuous High Resolution	19
13	Quy trình giao tiếp tổng thể giữa IP DHT11 và cảm biến	27
14	IP gửi tín hiệu khởi động và phản hồi từ DHT11	27
15	Biểu diễn truyền dữ liệu bit “0” của DHT11	28
16	Biểu diễn truyền dữ liệu bit “1” của DHT11	28
17	Sơ đồ máy trạng thái hữu hạn (FSM) điều khiển IP DHT11	29
18	Sơ đồ khối Block Design của hệ thống	35
19	Cấu hình các cổng I/O của hệ thống	35
20	Kết quả đọc dữ liệu cảm biến hiển thị qua UART theo chu kỳ 3 giây	38

DANH SÁCH BẢNG

1	Thông số kỹ thuật của kit FPGA Arty A7-35T	2
2	Cấu hình chân module cảm biến ánh sáng BH1750	3
3	Sơ đồ chân cảm biến DHT11	5
4	Sơ đồ chân mô-đun DHT11	5
5	Thông số kỹ thuật của cảm biến DHT11	5
6	AXI Timer Register Map	7
7	So sánh giao thức AXI4 Full và AXI4-Lite	8
8	Bảng ánh xạ thanh ghi AXI4-Lite của IP đọc cảm biến	13
9	Thiết lập địa chỉ I ² C cho BH1750	18
10	Các chế độ đo chính của BH1750	18

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1 Mục tiêu đề tài

Mục tiêu của đề tài “Thiết kế IP đọc giá trị cảm biến theo chu kỳ” là nghiên cứu và xây dựng một hệ thống IP core hoạt động ổn định trên phần cứng số, nhằm thực hiện việc thu thập dữ liệu cảm biến một cách tự động theo chu kỳ định trước thông qua cơ chế định thời độc lập.

Trong đó, hệ thống được thiết kế với hai IP chính bao gồm IP Timer, đảm nhiệm chức năng tạo tín hiệu định thời chính xác để điều khiển chu kỳ lấy mẫu, và IP đọc giá trị cảm biến, có nhiệm vụ giao tiếp và thu thập dữ liệu từ cảm biến nhiệt độ độ ẩm DHT11 và cảm biến ánh sáng BH1750. Cách tổ chức này giúp hệ thống đảm bảo tính đồng bộ trong quá trình đo, đồng thời nâng cao độ tin cậy của dữ liệu đầu ra.

Thông qua quá trình thiết kế, mô phỏng và kiểm thử, đề tài hướng tới việc củng cố kiến thức về thiết kế IP trong lĩnh vực điện tử vi mạch, nâng cao khả năng tổ chức các khối chức năng và tích hợp hệ thống, đồng thời tạo nền tảng cho việc phát triển các ứng dụng đo lường, giám sát và điều khiển trong thực tế.

1.2 Cơ sở kỹ thuật và linh kiện

1.2.1 Nền tảng phần cứng: FPGA Arty A7-35T

Để đáp ứng yêu cầu thiết kế và triển khai các IP core đọc giá trị cảm biến theo chu kỳ, đồng thời đảm bảo khả năng xử lý song song và giao tiếp ngoại vi ổn định, đề tài lựa chọn bo mạch phát triển Arty A7-35T của Digilent làm nền tảng phần cứng trung tâm. Theo tài liệu chính thức từ Digilent [1], Arty A7 là nền tảng FPGA sẵn sàng sử dụng, được thiết kế xoay quanh dòng chip Artix-7 của Xilinx, hướng đến các ứng dụng xử lý số, hệ thống nhúng và điều khiển.

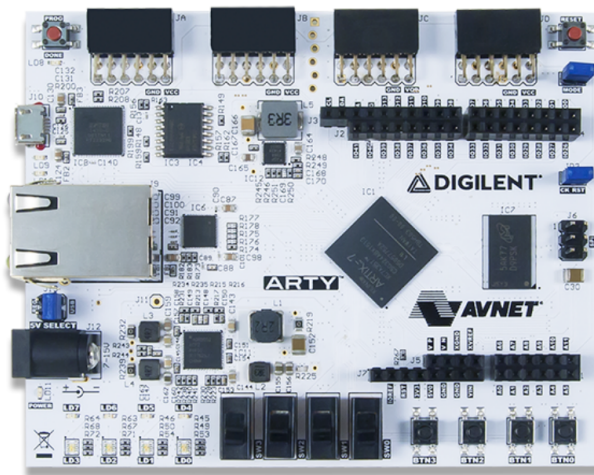


Figure 1: Bo mạch phát triển Arty A7-35T

Trung tâm của bo mạch là chip FPGA XC7A35TICSG324-1L, cung cấp các tài nguyên phần cứng quan trọng phục vụ trực tiếp cho việc thiết kế IP Timer và IP đọc cảm biến. FPGA sở hữu khoảng 5.200 logic slices, cho phép xây dựng các khối logic tổ hợp và tuần tự như bộ tạo xung định thời, máy trạng thái điều khiển và các khối giao tiếp cảm biến. Bên cạnh đó, chip còn tích hợp bộ nhớ Block RAM dung lượng lớn, hỗ trợ lưu trữ dữ liệu trung gian và kết quả đo từ cảm biến, giúp quá trình đọc dữ liệu theo chu kỳ diễn ra ổn định và liên tục.

Các thông số kỹ thuật chính của bo mạch Arty A7-35T được tóm tắt trong Bảng 1:

Table 1: Thông số kỹ thuật của kit FPGA Arty A7-35T

Nhóm	Thông số	Giá trị
FPGA	Chip FPGA	Artix-7 XC7A35TICSG324-1L
	Logic Slices	5.200
	Logic Cells	33.280
	Block RAM	1.800 Kbits
	DSP Slices	90
	Tần số nội bộ	>450 MHz
Clock & Memory	Clock hệ thống	100 MHz
	Bộ nhớ DDR3	256 MB @ 333 MHz
I/O & Giao tiếp	Cổng Pmod	4 cổng
	Giao tiếp	UART, SPI, I ² C
	Công tắc	4 switches
	Nút nhấn	4 buttons
	Đèn LED	4 user LEDs + 1 reset LED

Ngoài tài nguyên logic và bộ nhớ, FPGA Artix-7 trên Arty A7 còn được trang bị các khối DSP, hỗ trợ thực hiện các phép toán số học với tốc độ cao. Các khối này hữu ích trong quá trình xử lý và chuẩn hóa dữ liệu đo từ cảm biến nhiệt độ độ ẩm DHT11 cũng như xử lý dữ liệu ánh sáng thu được từ cảm biến BH1750. Tốc độ xung nhịp nội bộ cao của FPGA cho phép hệ thống đáp ứng tốt yêu cầu đọc dữ liệu theo chu kỳ ngắn và xử lý song song nhiều khối IP trong cùng một thời điểm.

Về khả năng giao tiếp, Arty A7 hỗ trợ đầy đủ các chuẩn phổ biến như UART, SPI và I²C. Trong phạm vi đề tài, giao tiếp I²C được sử dụng để xây dựng IP đọc cảm biến ánh sáng BH1750, trong khi cảm biến nhiệt độ độ ẩm DHT11 được giao tiếp thông qua giao thức một dây và xử lý hoàn toàn bằng logic số bên trong FPGA. Bo mạch cung cấp các cổng Pmod chuẩn, cho phép kết nối linh hoạt với các module cảm biến và mạch ngoại vi, phục vụ quá trình kiểm thử và đánh giá hệ thống.

Bên cạnh đó, Arty A7 còn tích hợp các phần tử nhập xuất cơ bản như công tắc, nút nhấn và đèn LED người dùng. Các phần tử này được sử dụng trong đề tài để hỗ trợ kiểm tra hoạt động của IP Timer, IP đọc cảm biến và quan sát trạng thái hệ thống trong quá trình vận

hành.

Với tài nguyên phần cứng phù hợp, khả năng mở rộng linh hoạt và sự hỗ trợ tốt từ cộng đồng phát triển, Arty A7-35T là nền tảng thích hợp để triển khai đề tài thiết kế IP đọc giá trị cảm biến theo chu kỳ, đồng thời tạo điều kiện thuận lợi cho việc mở rộng và phát triển các hệ thống đo lường và giám sát trong tương lai.

1.2.2 Module cảm biến ánh sáng BH1750

BH1750 [2] là module cảm biến ánh sáng dạng số, được sử dụng phổ biến trong các thiết bị điện tử tiêu dùng như điện thoại di động nhằm tự động điều chỉnh độ sáng màn hình theo điều kiện ánh sáng môi trường. Cảm biến có khả năng đo trực tiếp cường độ ánh sáng và cung cấp kết quả dưới dạng số với đơn vị lux, độ phân giải cao và giá trị đo tối đa lên đến 65.535 lx. Do dữ liệu đầu ra đã được số hóa, hệ thống xử lý không cần thực hiện các phép chuyển đổi tương tự sang số phức tạp.

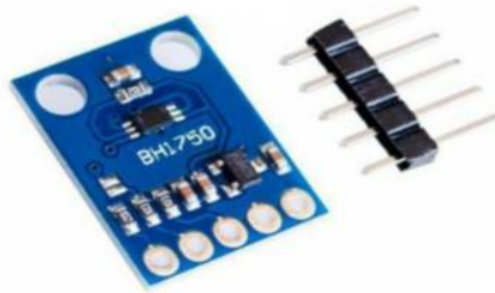


Figure 2: Module cảm biến ánh sáng BH1750

Module BH1750 sử dụng giao thức truyền thông I^2C để giao tiếp với hệ thống xử lý trung tâm và có năm chân kết nối chính, bao gồm chân cấp nguồn VCC, chân nối đất GND, chân SCL cung cấp xung clock, chân SDA truyền dữ liệu và chân ADDR dùng để lựa chọn địa chỉ thiết bị khi có nhiều module cùng kết nối trên một bus I^2C . Cấu hình chân của module được trình bày trong Bảng 2.

Table 2: Cấu hình chân module cảm biến ánh sáng BH1750

Số chân	Tên chân	Mô tả
1	VCC	Chân cấp nguồn từ 2.4V đến 3.6V (thường sử dụng 3.3V)
2	GND	Chân nối đất
3	SCL	Chân xung clock của giao thức I^2C
4	SDA	Chân truyền dữ liệu của giao thức I^2C
5	ADDR	Chân chọn địa chỉ thiết bị

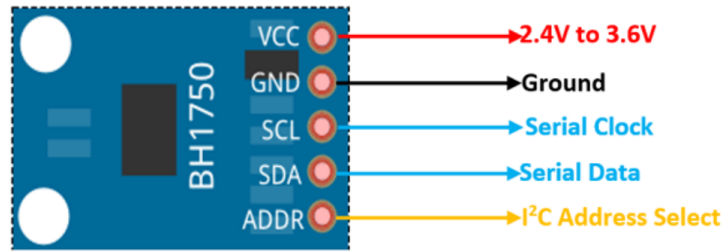


Figure 3: Sơ đồ cấu hình chân BH1750

Module BH1750 hoạt động trong dải điện áp từ 2.4V đến 3.6V, với mức tiêu thụ dòng điện rất thấp, khoảng 0.12 mA. Khoảng giá trị đo ánh sáng từ 1 lx đến 65.535 lx, với sai số đo khoảng $\pm 20\%$. Bên trong IC BH1750FVI tích hợp diode quang có độ nhạy gần với đặc tính của mắt người, cùng bộ chuyển đổi tương tự sang số ADC, cho phép chuyển đổi trực tiếp cường độ ánh sáng thành dữ liệu số và truyền về hệ thống xử lý thông qua bus I²C.

Trong quá trình sử dụng, module thường hoạt động ở điện áp 3.3V và các đường tín hiệu I²C cần sử dụng điện trở kéo lên với giá trị điển hình 4.7 k Ω . Nhờ cấu trúc đo tối ưu và khả năng lọc nhiễu tốt, kết quả đo của cảm biến ít bị ảnh hưởng bởi bức xạ hồng ngoại và các nguồn sáng không mong muốn.

Với các đặc tính như độ chính xác cao, mức tiêu thụ năng lượng thấp và giao tiếp đơn giản, module cảm biến ánh sáng BH1750 được ứng dụng rộng rãi trong các hệ thống tự động điều chỉnh độ sáng màn hình, hệ thống chiếu sáng thông minh, điều khiển bật tắt đèn theo điều kiện ánh sáng môi trường cũng như các hệ thống đo lường và giám sát ánh sáng trong thực tế.

1.2.3 Cảm biến nhiệt độ và độ ẩm DHT11

DHT11 [3] là cảm biến nhiệt độ và độ ẩm phổ biến, được tích hợp một điện trở nhiệt NTC chuyên dụng để đo nhiệt độ cùng với một bộ vi điều khiển 8 bit bên trong nhằm xử lý và xuất dữ liệu. Các giá trị nhiệt độ và độ ẩm được truyền ra ngoài dưới dạng dữ liệu nối tiếp, giúp cảm biến dễ dàng giao tiếp với nhiều loại vi điều khiển khác nhau.

Cảm biến DHT11 đã được hiệu chuẩn sẵn tại nhà máy, do đó việc sử dụng và tích hợp vào hệ thống nhúng trở nên đơn giản, ổn định và đáng tin cậy.

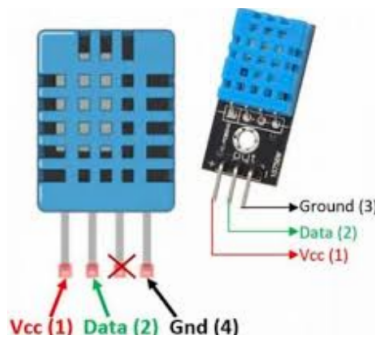


Figure 4: DHT11 thực tế

Table 3: Sơ đồ chân cảm biến DHT11

STT	Tên chân	Mô tả
1	VCC	Nguồn cấp từ 3.5 V đến 5.5 V
2	Data	Dữ liệu nối tiếp chứa thông tin nhiệt độ và độ ẩm
3	NC	Không kết nối
4	GND	Nối đất

Cảm biến DHT11 (dạng rời 4 chân)

Table 4: Sơ đồ chân mô-đun DHT11

STT	Tên chân	Mô tả
1	VCC	Nguồn cấp từ 3.5 V đến 5.5 V
2	Data	Dữ liệu nối tiếp nhiệt độ và độ ẩm
3	GND	Nối đất

Mô-đun DHT11 (dạng module) Bảng dưới đây là thông số kỹ thuật của DHT11:

Table 5: Thông số kỹ thuật của cảm biến DHT11

Thông số	Giá trị
Điện áp hoạt động	3.5 V – 5.5 V
Dòng hoạt động	0.3 mA (đo), 60 μ A (chế độ chờ)
Chuẩn giao tiếp	Dữ liệu nối tiếp
Phạm vi nhiệt độ	0°C – 50°C
Phạm vi độ ẩm	20% – 90% RH
Độ phân giải	16 bit cho nhiệt độ và độ ẩm
Độ chính xác	$\pm 1^\circ\text{C}$, $\pm 1\%$ RH

Cảm biến DHT11 có thể được cung cấp dưới hai dạng: cảm biến rời và mô-đun. Về mặt chức năng và hiệu suất đo, cả hai đều tương đương nhau. Tuy nhiên, mô-đun DHT11 đã được tích hợp sẵn điện trở kéo lên và tụ lọc, giúp việc kết nối trở nên thuận tiện hơn.

Trong khi đó, đối với cảm biến DHT11 dạng rời, người dùng cần bổ sung các linh kiện này bên ngoài nếu hệ thống yêu cầu, nhằm đảm bảo tín hiệu dữ liệu ổn định.

Để khởi tạo quá trình truyền dữ liệu, vi điều khiển cần kéo chân dữ liệu xuống mức thấp trong một khoảng thời gian ngắn, sau đó thả lên mức cao. Cảm biến sẽ phản hồi và gửi dữ liệu theo đúng biểu đồ thời gian được mô tả trong tài liệu kỹ thuật của DHT11.

Ứng dụng của DHT11 là: Đo nhiệt độ và độ ẩm môi trường, đài thời tiết mini, hệ thống kiểm soát khí hậu tự động, giám sát môi trường

1.2.4 Timer tích hợp trên FPGA

Trong các hệ thống nhúng triển khai trên FPGA, Timer là khối phần cứng quan trọng dùng để tạo mốc thời gian chính xác nhằm điều khiển các tác vụ hoạt động theo chu kỳ. Việc sử dụng Timer phần cứng giúp hệ thống tránh phụ thuộc vào các hàm trễ bằng phần mềm, từ đó giảm tải cho bộ xử lý và nâng cao độ ổn định của hệ thống.

Đối với các ứng dụng giám sát môi trường, yêu cầu đọc dữ liệu cảm biến theo chu kỳ cố định là rất phổ biến. Do đó, Timer được sử dụng để làm cơ sở định thời, đảm bảo quá trình lấy mẫu diễn ra đều đặn và nhất quán.

AXI Timer/Counter là IP core tiêu chuẩn do AMD Xilinx cung cấp [4], được tích hợp sẵn trong Vivado Design Suite. IP này giao tiếp với bộ xử lý thông qua bus AXI4-Lite, cho phép phần mềm cấu hình và điều khiển Timer thông qua các thanh ghi ảnh xạ bộ nhớ.

AXI Timer bao gồm hai bộ đếm độc lập, mỗi bộ đếm có độ rộng 32-bit, hỗ trợ nhiều chế độ hoạt động khác nhau như: Generate mode, Capture mode, Pulse Width Modulation (PWM), Cascade mode (kết hợp hai Timer thành bộ đếm 64-bit).

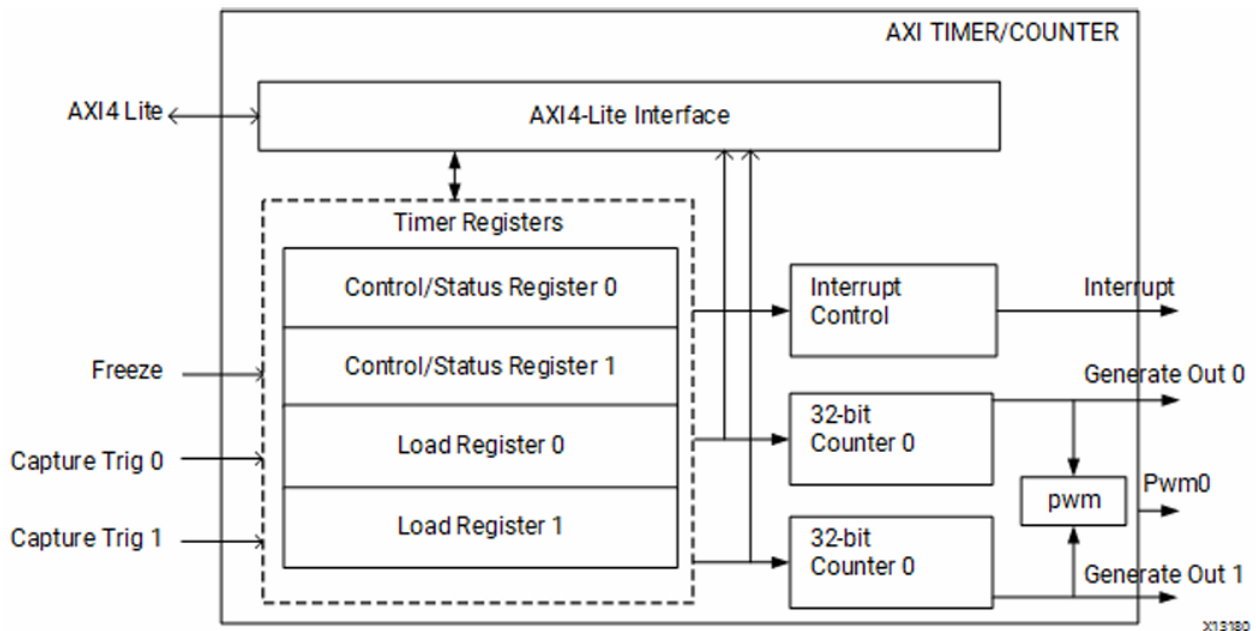


Figure 5: Sơ đồ khối AXI Timer/Counter IP Core

Như thể hiện trong hình trên, AXI Timer bao gồm các khối chính như giao diện AXI4-Lite, hệ thống thanh ghi điều khiển và các bộ đếm phần cứng. Thông qua giao diện AXI, bộ xử lý có thể ghi và đọc các thanh ghi của Timer để cấu hình chế độ hoạt động, giá trị đếm và trạng thái Timer.

Việc điều khiển AXI Timer được thực hiện thông qua tập các thanh ghi ánh xạ bộ nhớ. Mỗi Timer bao gồm ba thanh ghi chính:

TCSR (Timer Control and Status Register): cấu hình chế độ hoạt động, hướng đếm, tự động nạp lại và trạng thái Timer

TLR (Timer Load Register): lưu giá trị nạp ban đầu cho bộ đếm

TCR (Timer Counter Register): lưu giá trị đếm hiện tại của Timer

Table 6: AXI Timer Register Map

Address Offset	Register Name	Description
0x00	TCSR0	Timer 0 Control and Status Register
0x04	TLR0	Timer 0 Load Register
0x08	TCR0	Timer 0 Counter Register
0x0C–0x0F	RSVD	Reserved
0x10	TCSR1	Timer 1 Control and Status Register
0x14	TLR1	Timer 1 Load Register
0x18	TCR1	Timer 1 Counter Register
0x1C–0x1F	RSVD	Reserved

Trong đề tài này, AXI Timer được cấu hình hoạt động ở Generate mode. Ở chế độ này, giá trị trong thanh ghi TLR sẽ được nạp vào bộ đếm. Khi Timer được kích hoạt, bộ đếm sẽ tăng hoặc giảm tùy theo cấu hình. Khi bộ đếm đạt đến giá trị giới hạn, Timer sẽ sinh ra sự kiện hết hạn (timeout).

Nếu chế độ Auto-reload được kích hoạt, Timer sẽ tự động nạp lại giá trị ban đầu và tiếp tục đếm, từ đó tạo ra chu kỳ thời gian lặp lại một cách liên tục.

Chế độ này đặc biệt phù hợp với các ứng dụng cần thực hiện tác vụ định kỳ, chẳng hạn như đọc dữ liệu cảm biến theo chu kỳ cố định.

1.3 Giới thiệu về giao tiếp AXI

1.3.1 Tổng quan AXI và AMBA

AMBA (Advanced Microcontroller Bus Architecture): Là một kiến trúc bus giao tiếp nội bộ dùng trong các hệ thống trên chip (SoC).

AXI (Advanced eXtensible Interface) [5]: Là một giao thức trong hệ kiến trúc AMBA, hỗ trợ các thiết kế hệ thống hiệu suất cao và tần số cao.

Giao thức AXI (Advanced eXtensible Interface) được thiết kế nhằm đáp ứng các hệ thống số yêu cầu băng thông cao và độ trễ thấp. Nhờ kiến trúc tối ưu, AXI cho phép hệ thống hoạt động ở tần số cao mà không cần sử dụng các khối cầu nối phức tạp, từ đó giúp giảm độ trễ và đơn giản hóa thiết kế phần cứng. Bên cạnh đó, giao thức này có khả năng đáp ứng yêu cầu giao tiếp của nhiều loại khối chức năng khác nhau, từ các IP xử lý, bộ điều khiển ngoại vi cho đến bộ điều khiển bộ nhớ.

AXI đặc biệt phù hợp với các bộ điều khiển bộ nhớ có độ trễ truy cập ban đầu lớn, do giao thức cho phép tách biệt rõ ràng giữa quá trình cấp phát địa chỉ và quá trình truyền dữ liệu. Điều này giúp hệ thống có thể tiếp tục xử lý các tác vụ khác trong khi chờ dữ liệu phản hồi, qua đó nâng cao hiệu năng tổng thể. Ngoài ra, AXI còn cung cấp tính linh hoạt cao trong việc xây dựng kiến trúc liên kết giữa các khối IP, cho phép dễ dàng mở rộng hoặc thay đổi cấu trúc hệ thống khi cần thiết. Một ưu điểm quan trọng khác của AXI là khả năng tương thích ngược với các giao diện bus thế hệ trước như AHB và APB, giúp thuận lợi trong việc tích hợp các IP cũ vào hệ thống mới.

Về mặt kiến trúc giao tiếp, AXI phân tách rõ ràng giữa pha địa chỉ và điều khiển với pha truyền dữ liệu, giúp tăng khả năng xử lý song song và giảm xung đột trên bus. Giao thức hỗ trợ truyền dữ liệu không cần hàng thông qua cơ chế byte strobe, cho phép ghi và đọc dữ liệu linh hoạt theo từng byte. Các giao dịch trên AXI được thực hiện theo cơ chế burst, trong đó chỉ cần cung cấp địa chỉ bắt đầu, còn dữ liệu sẽ được truyền liên tiếp theo số lượng xác định trước.

AXI sử dụng các kênh đọc và ghi riêng biệt, cho phép thực hiện các thao tác truyền dữ liệu độc lập và hỗ trợ hiệu quả cho các cơ chế truy cập bộ nhớ trực tiếp như DMA với chi phí phần cứng thấp. Ngoài ra, giao thức cho phép phát sinh nhiều yêu cầu địa chỉ chưa hoàn tất tại cùng một thời điểm, cũng như hỗ trợ hoàn tất giao dịch không theo thứ tự, từ đó tối ưu hiệu năng trong các hệ thống có nhiều tác vụ song song. Cuối cùng, kiến trúc AXI cho phép dễ dàng chèn thêm các tầng thanh ghi trung gian nhằm đáp ứng yêu cầu đồng thời gian (timing closure), đặc biệt quan trọng đối với các thiết kế hoạt động ở tần số cao.

1.3.2 AXI4-Lite và lý do lựa chọn

Trong hệ thống thiết kế IP trên FPGA, giao thức AXI4 cung cấp một chuẩn giao tiếp mạnh mẽ và linh hoạt giữa các khối xử lý và ngoại vi. Tuy nhiên, tùy theo đặc tính dữ liệu và mục đích sử dụng của từng IP, việc lựa chọn biến thể AXI phù hợp là yếu tố quan trọng nhằm tối ưu tài nguyên phần cứng và đơn giản hóa thiết kế.

AXI4 đầy đủ được thiết kế cho các ứng dụng yêu cầu truyền dữ liệu khối lớn với thông lượng cao, điển hình như truy cập bộ nhớ DDR hoặc Flash. Giao thức này hỗ trợ các giao dịch burst với độ dài lên đến 256 lần truyền, cho phép nhiều kiểu burst khác nhau như FIXED, INCR và WRAP, đồng thời hỗ trợ các cơ chế truy cập đọc quyền. Bên cạnh đó, AXI4 Full cho phép độ rộng dữ liệu rất lớn, có thể mở rộng từ 8-bit đến 1024-bit, phù hợp với các hệ thống xử lý hiệu năng cao.

Ngược lại, AXI4-Lite là một tập con được đơn giản hóa của AXI4, trong đó mỗi giao dịch chỉ bao gồm một lần truyền dữ liệu duy nhất và không hỗ trợ các cơ chế burst phức tạp. Kiểu truyền dữ liệu mặc định của AXI4-Lite là INCR, nhưng bị giới hạn ở một transfer, giúp giảm đáng kể số lượng tín hiệu điều khiển cũng như logic xử lý bên trong IP. Độ rộng dữ liệu của AXI4-Lite thường được sử dụng ở mức 32-bit hoặc 64-bit, phù hợp với việc ánh xạ các thanh ghi điều khiển và thanh ghi dữ liệu có kích thước nhỏ.

Table 7: So sánh giao thức AXI4 Full và AXI4-Lite

Đặc điểm	AXI4 (Full)	AXI4-Lite
Độ dài Burst	Hỗ trợ burst lên đến 256 lần truyền	Luôn cố định 1 lần truyền
Kiểu Burst	FIXED, INCR, WRAP	INCR (1 lần truyền)
Độ rộng dữ liệu	8-bit đến 1024-bit	Thường 32-bit hoặc 64-bit
Truy cập đọc quyền	Có hỗ trợ	Không hỗ trợ
Ứng dụng	Truy cập bộ nhớ tốc độ cao (DDR, Flash)	Cấu hình và đọc thanh ghi IP

Sự khác biệt cơ bản giữa AXI4 Full và AXI4-Lite được tóm tắt trong Bảng 7. Từ bảng so sánh có thể thấy rằng AXI4-Lite không hướng tới việc truyền dữ liệu tốc độ cao hay xử

lý bộ nhớ khối lớn, mà tập trung vào giao tiếp đơn giản, ổn định và dễ tích hợp cho các IP ngoại vi.

Đối với đề tài này, các IP đọc cảm biến BH1750 và DHT11 chỉ cung cấp dữ liệu đo môi trường có dung lượng nhỏ và được cập nhật theo chu kỳ định thời xác định trước bởi IP Timer. Do đó, việc sử dụng AXI4-Lite để ánh xạ các thanh ghi dữ liệu cảm biến và thanh ghi điều khiển là lựa chọn phù hợp, vừa đáp ứng đầy đủ yêu cầu chức năng, vừa giảm độ phức tạp thiết kế so với việc sử dụng AXI4 Full.

1.3.3 Cơ chế bắt tay VALID/READY

Trong giao thức AXI, việc truyền dữ liệu giữa các khối chức năng được điều khiển thông qua cơ chế bắt tay nhằm đảm bảo tính đồng bộ và độ tin cậy của quá trình giao tiếp. Đối với AXI4-Lite, cơ chế truyền dữ liệu được sử dụng là hình thức Ready, trong đó mỗi kênh giao tiếp đều bao gồm cặp tín hiệu VALID và READY.

Theo cơ chế này, thiết bị phát sẽ khẳng định tín hiệu VALID khi có địa chỉ, dữ liệu hoặc thông tin điều khiển cần truyền đi, trong khi thiết bị nhận sẽ khẳng định tín hiệu READY khi sẵn sàng tiếp nhận dữ liệu. Một lần truyền được xem là hợp lệ khi cả hai tín hiệu VALID và READY cùng ở mức logic cao tại cạnh lên của xung nhịp. Cơ chế bắt tay này cho phép hai phía chủ động kiểm soát tốc độ truyền, đồng thời giúp hệ thống hoạt động ổn định ngay cả khi tồn tại sự chênh lệch về tốc độ xử lý giữa các khối IP.

AXI4-Lite không hỗ trợ hình thức truyền dữ liệu theo cơ chế Credited, do đó toàn bộ các kênh đọc và ghi trong giao diện đều sử dụng cơ chế Ready để điều khiển luồng dữ liệu. Đặc điểm này giúp đơn giản hóa logic điều khiển, giảm số lượng tín hiệu cần thiết và đặc biệt phù hợp với các IP ngoại vi có thông lượng thấp.

Trong phạm vi đề tài, các IP đọc cảm biến BH1750 và DHT11 sử dụng giao diện AXI4-Lite với cơ chế bắt tay VALID/READY để ánh xạ và truy xuất các thanh ghi dữ liệu cảm biến cũng như thanh ghi điều khiển. Việc lựa chọn cơ chế này đảm bảo tính đơn giản trong thiết kế IP, đồng thời đáp ứng đầy đủ yêu cầu đọc dữ liệu cảm biến theo chu kỳ trên nền tảng FPGA Arty A7-35T.

Các kịch bản bắt tay VALID/READY trong AXI4-Lite

Cơ chế bắt tay VALID/READY cho phép thiết bị truyền và thiết bị nhận hoạt động độc lập về mặt thời gian, trong khi vẫn đảm bảo dữ liệu được truyền một cách chính xác và đồng bộ. Tùy thuộc vào trạng thái sẵn sàng của từng phía, quá trình bắt tay có thể xảy ra theo các kịch bản khác nhau.

Kịch bản thứ nhất xảy ra khi tín hiệu VALID được khẳng định trước tín hiệu READY. Trong trường hợp này, thiết bị truyền đã có dữ liệu hợp lệ và đưa dữ liệu lên bus, đồng thời giữ tín hiệu VALID ở mức cao. Thiết bị nhận chưa sẵn sàng tiếp nhận dữ liệu nên tín hiệu READY vẫn ở mức thấp. Quá trình truyền chỉ thực sự diễn ra khi thiết bị nhận khẳng định READY ở một chu kỳ xung nhịp sau đó. Kịch bản này thường xuất hiện khi thiết bị truyền hoạt động nhanh hơn hoặc khi dữ liệu đã sẵn sàng trước khi thiết bị nhận hoàn tất các tác vụ nội bộ.

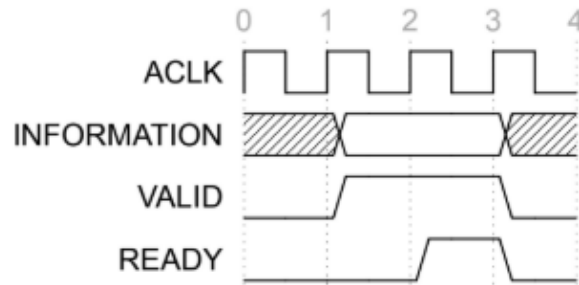


Figure 6: Kịch bản VALID được khẳng định trước READY

Kịch bản thứ hai xảy ra khi tín hiệu READY được khẳng định trước tín hiệu VALID. Khi đó, thiết bị nhận đã sẵn sàng tiếp nhận dữ liệu và đưa READY lên mức cao, trong khi thiết bị truyền chưa có dữ liệu hợp lệ nên tín hiệu VALID vẫn ở mức thấp. Quá trình truyền chỉ được thực hiện khi thiết bị truyền khẳng định VALID ở chu kỳ xung nhịp tiếp theo. Kịch bản này phản ánh trường hợp thiết bị nhận chủ động chờ dữ liệu từ phía thiết bị truyền.

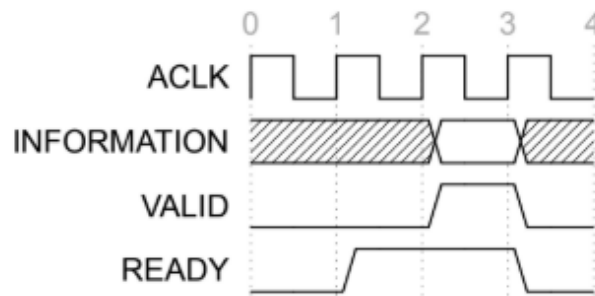


Figure 7: Kịch bản READY được khẳng định trước VALID

Kịch bản thứ ba là trường hợp cả hai tín hiệu VALID và READY cùng được khẳng định trong cùng một chu kỳ xung nhịp. Khi đó, quá trình bắt tay diễn ra ngay lập tức và dữ liệu được truyền thành công chỉ trong một chu kỳ clock. Đây là kịch bản tối ưu nhất về mặt hiệu năng, thường xuất hiện khi cả thiết bị truyền và thiết bị nhận đều ở trạng thái sẵn sàng tại cùng một thời điểm.

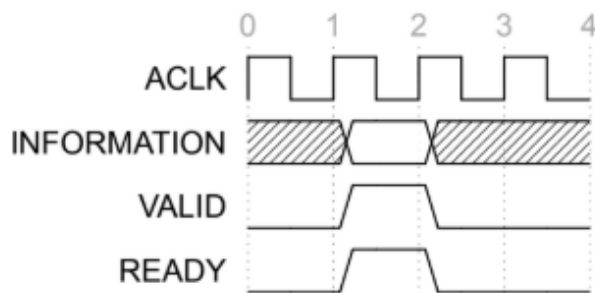


Figure 8: Kịch bản VALID và READY cùng được khẳng định trong một chu kỳ clock

Ba kịch bản bắt tay trên cho thấy cơ chế VALID/READY trong AXI4-Lite có tính linh hoạt cao, cho phép các khối IP hoạt động không đồng bộ về mặt thời gian mà không làm mất dữ liệu. Trong phạm vi đề tài, cơ chế này được sử dụng để đảm bảo việc đọc dữ liệu từ các IP cảm biến BH1750 và DHT11 diễn ra ổn định, ngay cả khi có sự khác biệt về tốc độ xử lý giữa IP cảm biến và khối xử lý trung tâm.

1.3.4 Quá trình đọc/ghi trong AXI4-Lite

Giao thức AXI định nghĩa các giao dịch (transaction) dùng để thực hiện việc đọc và ghi dữ liệu giữa thiết bị chủ (Manager) và thiết bị phụ (Subordinate). Mỗi giao dịch được thực hiện thông qua việc truyền dữ liệu và thông tin điều khiển trên các kênh độc lập, cho phép tăng khả năng xử lý song song và nâng cao hiệu năng hệ thống. Trong AXI4-Lite, các giao dịch này được đơn giản hóa nhằm phục vụ cho việc truy cập và cấu hình các thanh ghi ánh xạ bộ nhớ.

AXI sử dụng năm kênh truyền cơ bản bao gồm kênh yêu cầu ghi (AW), kênh dữ liệu ghi (W), kênh phản hồi ghi (B), kênh yêu cầu đọc (AR) và kênh dữ liệu đọc (R). Các kênh yêu cầu mang theo thông tin địa chỉ và điều khiển, trong khi các kênh dữ liệu thực hiện việc truyền dữ liệu giữa Manager và Subordinate.

Quá trình ghi trong AXI4-Lite

Quá trình ghi dữ liệu trong AXI4-Lite được thực hiện thông qua ba kênh độc lập là kênh ghi địa chỉ, kênh ghi dữ liệu và kênh phản hồi ghi.

Đầu tiên, thiết bị Manager gửi địa chỉ ghi và các thông tin điều khiển liên quan thông qua kênh ghi địa chỉ AW bằng cách khẳng định tín hiệu AWVALID. Khi thiết bị Subordinate sẵn sàng tiếp nhận địa chỉ, nó sẽ khẳng định tín hiệu AWREADY và quá trình bắt tay trên kênh này được hoàn tất.

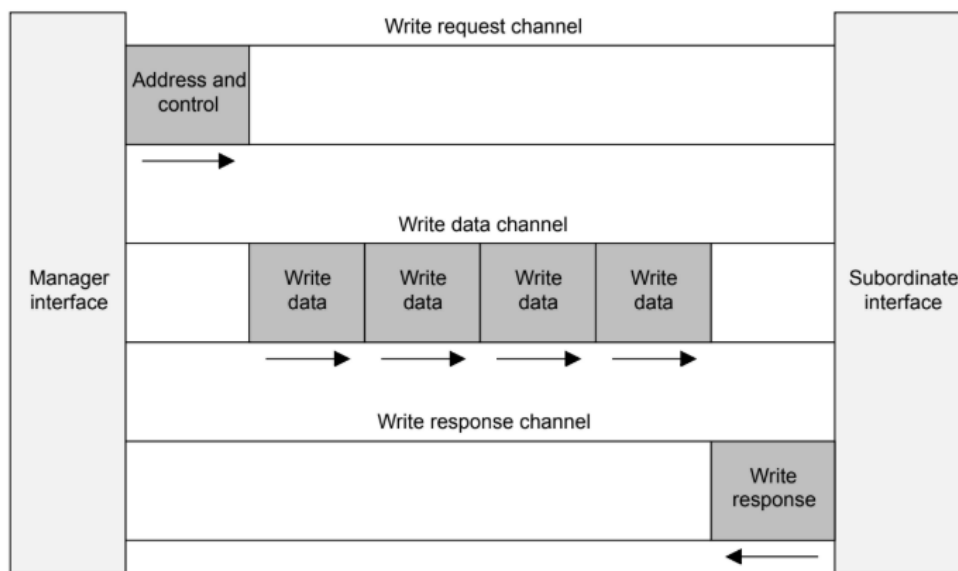


Figure 9: Channel architecture of writes

Song song hoặc sau đó, Manager gửi dữ liệu cần ghi thông qua kênh dữ liệu ghi W bằng cách khẳng định tín hiệu WVALID. Subordinate khẳng định tín hiệu WREADY khi có thể tiếp nhận dữ liệu. Dữ liệu ghi được truyền thành công khi cả hai tín hiệu WVALID và WREADY cùng ở mức logic cao tại cạnh lên của xung nhịp.

Sau khi Subordinate đã nhận đầy đủ cả địa chỉ ghi và dữ liệu ghi, nó sẽ gửi tín hiệu phản hồi hoàn tất giao dịch thông qua kênh phản hồi ghi B bằng cách khẳng định tín hiệu BVALID. Manager khẳng định tín hiệu BREADY để chấp nhận phản hồi này. Quá trình ghi chỉ được xem là hoàn tất khi quá trình bắt tay trên kênh B diễn ra thành công.

Đối với AXI4-Lite, thiết bị Subordinate bắt buộc phải đợi cho đến khi cả địa chỉ và dữ liệu ghi đều được nhận thành công, tức là các tín hiệu AWVALID, AWREADY, WVALID và WREADY đều đã được khẳng định, trước khi phát tín hiệu BVALID phản hồi ghi. Quy tắc này đảm bảo tính đơn giản và tính nhất quán trong cơ chế ghi của AXI4-Lite.

Quá trình đọc trong AXI4-Lite

Quá trình đọc trong AXI4-Lite được thực hiện thông qua hai kênh là kênh yêu cầu đọc AR và kênh dữ liệu đọc R.

Thiết bị Manager gửi địa chỉ cần đọc thông qua kênh AR bằng cách khẳng định tín hiệu ARVALID. Khi Subordinate sẵn sàng tiếp nhận yêu cầu đọc, nó sẽ khẳng định tín hiệu ARREADY và hoàn tất quá trình bắt tay trên kênh yêu cầu đọc.

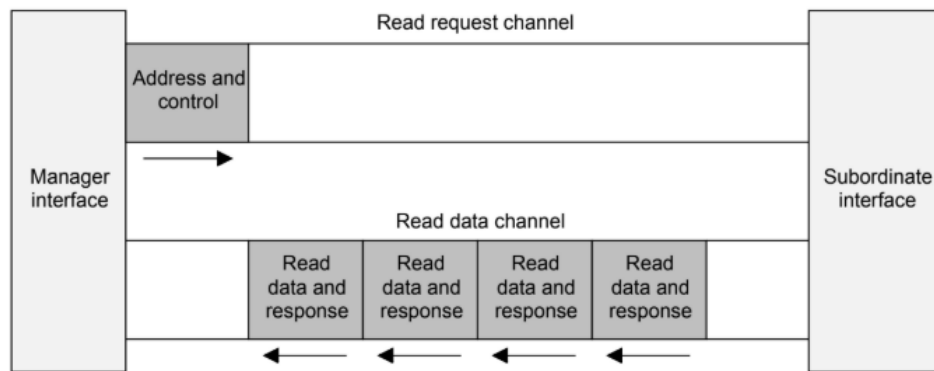


Figure 10: Channel architecture of reads

Sau khi nhận được địa chỉ đọc hợp lệ, Subordinate truy xuất dữ liệu tương ứng và gửi dữ liệu này về cho Manager thông qua kênh dữ liệu đọc R. Subordinate khẳng định tín hiệu RVALID để báo hiệu dữ liệu hợp lệ, trong khi Manager khẳng định tín hiệu RREADY khi sẵn sàng nhận dữ liệu. Giao dịch đọc được hoàn tất khi cả hai tín hiệu RVALID và RREADY cùng ở mức logic cao tại cạnh lên của xung nhịp.

Trong AXI4-Lite, dữ liệu đọc và phản hồi đọc được truyền đồng thời trên cùng kênh R, giúp đơn giản hóa giao thức và giảm số lượng tín hiệu cần thiết so với AXI4 đầy đủ.

1.3.5 Ánh xạ thanh ghi AXI4-Lite cho IP đọc cảm biến

Với IP đọc cảm biến (BH1750 hoặc DHT11) sử dụng giao diện AXI4-Lite, dữ liệu và các thông tin điều khiển được ánh xạ vào các thanh ghi nội bộ của IP. Mỗi thanh ghi tương ứng với một địa chỉ trong không gian địa chỉ AXI, cho phép bộ xử lý trung tâm truy xuất thông qua các giao dịch đọc/ghi chuẩn AXI4-Lite.

Table 8: Bảng ánh xạ thanh ghi AXI4-Lite của IP đọc cảm biến

Offset	Tên thanh ghi	Quyền truy cập	Chức năng
0x00	SLV_REG0	Read-only	Chứa dữ liệu cảm biến. BH1750: giá trị cường độ ánh sáng (lux) DHT11: có thể ánh xạ nhiệt độ hoặc độ ẩm
0x04	SLV_REG1	Read/Write	Thanh ghi điều khiển IP
0x08	SLV_REG2	Read/Write	Thanh ghi cấu hình mở rộng hoặc dự phòng cho các chức năng bổ sung.
0x0C	SLV_REG3	Read/Write	Thanh ghi trạng thái hoặc thanh ghi dự phòng cho việc mở rộng hệ thống.

Các thanh ghi của IP được ánh xạ liên tiếp trong không gian địa chỉ AXI4-Lite với bước địa chỉ 4 byte, tương ứng với độ rộng dữ liệu 32-bit. Thanh ghi SLV_REG0 được thiết kế ở chế độ chỉ đọc nhằm lưu trữ dữ liệu cảm biến, đảm bảo phần mềm không thể ghi đè lên dữ liệu đo. Các thanh ghi còn lại cho phép đọc/ghi và được sử dụng cho mục đích điều khiển, cấu hình hoặc mở rộng chức năng của IP trong tương lai.

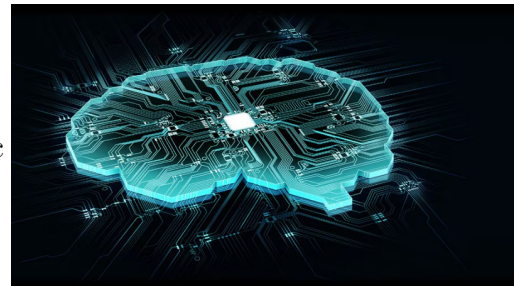
1.4 Công nghệ phần mềm

1.4.1 Vivado

AMD Vivado™ [6] là phần mềm thiết kế dành cho các SoC và FPGA thích ứng của AMD. Nó bao gồm: Nhập thiết kế, Tổng hợp, Đặt và Định tuyến, các công cụ Kiểm tra/Mô phỏng. Tìm hiểu cách các tính năng nâng cao trong phần mềm thiết kế Vivado giúp các nhà thiết kế phần cứng giảm thời gian biên dịch và số lần lặp lại thiết kế, đồng thời ước tính điện năng chính xác hơn cho các SoC và FPGA thích ứng của AMD.

Thiết kế và triển khai:

- Nhập liệu bằng VHDL/Verilog, tích hợp IP và thực hiện tổng hợp, đặt tuyến.

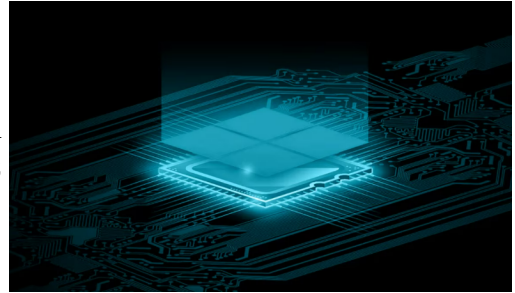


Xác minh chức năng:

- Mô phỏng và kiểm chứng thiết kế nhằm đảm bảo tính đúng đắn trước khi nạp xuống phần cứng.

**Gỡ lỗi phần cứng:**

- Sử dụng Vivado ChipScope để theo dõi và phân tích tín hiệu logic trong quá trình hệ thống hoạt động.



1.4.2 Vitis

Vitis là bộ công cụ phát triển phần mềm cho FPGA và SoC, được xây dựng trên cơ sở hợp nhất các công cụ trước đây của Xilinx như Xilinx SDK, Vivado HLS và SDSoC. Vitis cho phép phát triển ứng dụng C/C++ chạy trên bộ xử lý nhúng, đồng thời hỗ trợ chuyển đổi và tăng tốc các khối xử lý phần mềm sang phần cứng FPGA, giúp đơn giản hóa quy trình thiết kế hệ thống phần cứng – phần mềm đồng thiết kế.

Mặc dù Vivado và Vitis cùng được sử dụng trong thiết kế hệ thống dựa trên FPGA, hai môi trường này đại diện cho những cách tiếp cận khác nhau trong quy trình phát triển. Sự khác biệt không chỉ nằm ở ngôn ngữ sử dụng mà còn ở tư duy thiết kế giữa phần cứng song song và phần mềm tuần tự.

Cả Vivado và Vitis đều có khả năng tạo ra các thiết kế phần cứng song song cao trên kiến trúc FPGA. Tuy nhiên, Vitis mở rộng thêm khả năng xử lý các tác vụ tuần tự trên bộ xử lý nhúng, từ đó hỗ trợ mô hình đồng thiết kế phần cứng và phần mềm. Mỗi phương pháp đều tồn tại những thách thức riêng trong quá trình phát triển.

Trong thiết kế phần cứng song song, một khối xử lý có hiệu năng thấp có thể ảnh hưởng đến toàn bộ hệ thống, ngay cả khi khối đó chỉ được sử dụng ở mức hạn chế. Điều này có thể dẫn đến việc phải giảm tần số xung nhịp của toàn bộ thiết kế. Ngược lại, trong thiết kế phần mềm, các vấn đề về hiệu năng thường mang tính cục bộ và ít khi ảnh hưởng nghiêm trọng đến toàn bộ chương trình.

Việc gỡ lỗi trong thiết kế phần cứng thuần túy cũng phức tạp hơn so với phần mềm. Người thiết kế không thể tùy ý quan sát các tín hiệu nội bộ nếu các mạch gỡ lỗi chưa được tích hợp từ trước. Các công cụ như bộ phân tích logic bên trong hoặc bên ngoài FPGA cần được cấu hình và kết nối cụ thể, làm tăng thời gian phát triển. Tuy nhiên, các công cụ mô phỏng phần cứng trong Vivado vẫn hỗ trợ hiệu quả trong việc phát hiện và khắc phục lỗi.

Ngược lại, quá trình gỡ lỗi phần mềm trong môi trường Vitis linh hoạt và thuận tiện hơn. Khi chương trình được biên dịch ở chế độ gỡ lỗi, người phát triển có thể kiểm tra các biến, vùng nhớ và luồng thực thi mà không cần xác định trước trong quá trình thiết kế. Ngoài

ra, Vitis còn hỗ trợ mô phỏng và giả lập các khối phần cứng được mô tả bằng ngôn ngữ lập trình, giúp rút ngắn chu trình phát triển và kiểm thử hệ thống.

Trong hệ thống này, Vivado được sử dụng để thiết kế và cấu hình phần cứng trên FPGA, bao gồm bộ xử lý nhúng và các khối giao tiếp ngoại vi. Trong khi đó, Vitis được sử dụng để phát triển chương trình nhúng bằng ngôn ngữ C chạy trên bộ xử lý, thực hiện toàn bộ logic điều khiển và xử lý của trò chơi. Sự kết hợp giữa Vivado và Vitis cho phép xây dựng một hệ thống nhúng hoàn chỉnh, kết hợp chặt chẽ giữa phần cứng và phần mềm.

CHƯƠNG 2: THIẾT IP ĐỌC CẢM BIẾN

2.1 Tổng quan kiến trúc hệ thống

Hệ thống được thiết kế với mục tiêu thu thập dữ liệu cảm biến một cách tự động theo chu kỳ định trước thông qua việc sử dụng các IP core chuyên dụng hoạt động độc lập trên nền tảng phần cứng số. Kiến trúc hệ thống bao gồm hai khối IP chính là IP Timer và IP đọc giá trị cảm biến, trong đó IP Timer đảm nhiệm chức năng tạo tín hiệu định thời, còn IP đọc cảm biến thực hiện chức năng giao tiếp và thu thập dữ liệu từ cảm biến để cung cấp cho hệ thống xử lý trung tâm.

IP Timer được sử dụng để tạo chu kỳ hoạt động cho toàn bộ hệ thống bằng cách chia tần xung clock đầu vào nhằm sinh ra tín hiệu kích hoạt theo khoảng thời gian xác định. Tín hiệu này được dùng làm điều kiện khởi động quá trình đọc dữ liệu cảm biến, qua đó đảm bảo việc lấy mẫu diễn ra một cách tuần hoàn, ổn định và không phụ thuộc vào sự can thiệp của phần mềm.

IP đọc giá trị cảm biến nhận tín hiệu kích hoạt từ IP Timer và thực hiện quá trình giao tiếp với các cảm biến tương ứng. Trong phạm vi đề tài, IP được thiết kế để hỗ trợ hai loại cảm biến gồm cảm biến nhiệt độ và độ ẩm DHT11 và cảm biến ánh sáng BH1750. Cảm biến DHT11 sử dụng giao thức truyền dữ liệu nối tiếp một dây, trong đó IP thực hiện quá trình khởi tạo, nhận chuỗi dữ liệu 40 bit và tách các giá trị nhiệt độ và độ ẩm theo đúng định dạng của cảm biến. Cảm biến BH1750 được giao tiếp thông qua bus I²C và cung cấp dữ liệu số trực tiếp biểu diễn cường độ ánh sáng theo đơn vị lux.

Dữ liệu đo từ các cảm biến sau khi được xử lý sẽ được lưu trữ trong các thanh ghi nội bộ của IP đọc cảm biến và sẵn sàng cho các khối xử lý tiếp theo hoặc cho phép hệ thống phần mềm truy xuất thông qua giao tiếp bus AXI4-Lite. Việc tách biệt rõ ràng chức năng định thời và chức năng đọc cảm biến giúp hệ thống có cấu trúc mạch lạc, dễ mở rộng và thuận lợi cho việc tái sử dụng IP trong các ứng dụng khác.

Nguyên lý hoạt động tổng thể của hệ thống được xây dựng theo mô hình điều khiển theo chu kỳ, trong đó IP Timer đóng vai trò điều phối thời gian, còn IP đọc cảm biến thực hiện nhiệm vụ thu thập dữ liệu từ các cảm biến DHT11 và BH1750 theo tín hiệu kích hoạt. Cấu trúc này phù hợp với các hệ thống đo lường, giám sát môi trường và các ứng dụng điện tử vi mạch trên nền tảng FPGA.

2.2 Thiết kế IP đọc cảm biến

2.2.1 Thiết kế ip BH1750

Module cảm biến ánh sáng BH1750 được thiết kế dưới dạng một IP phần cứng nhằm thực hiện chức năng thu thập cường độ ánh sáng môi trường thông qua chuẩn giao tiếp I²C và cung cấp dữ liệu cho khối xử lý trung tâm của hệ thống. IP hoạt động độc lập với phần mềm điều khiển và tự động thực hiện quá trình đo theo chu kỳ, trong đó chu kỳ đo được xác định bởi IP Timer có sẵn trong hệ thống. Cách tiếp cận này giúp giảm sự phụ thuộc vào phần mềm, đồng thời nâng cao tính ổn định và khả năng mở rộng của hệ thống.

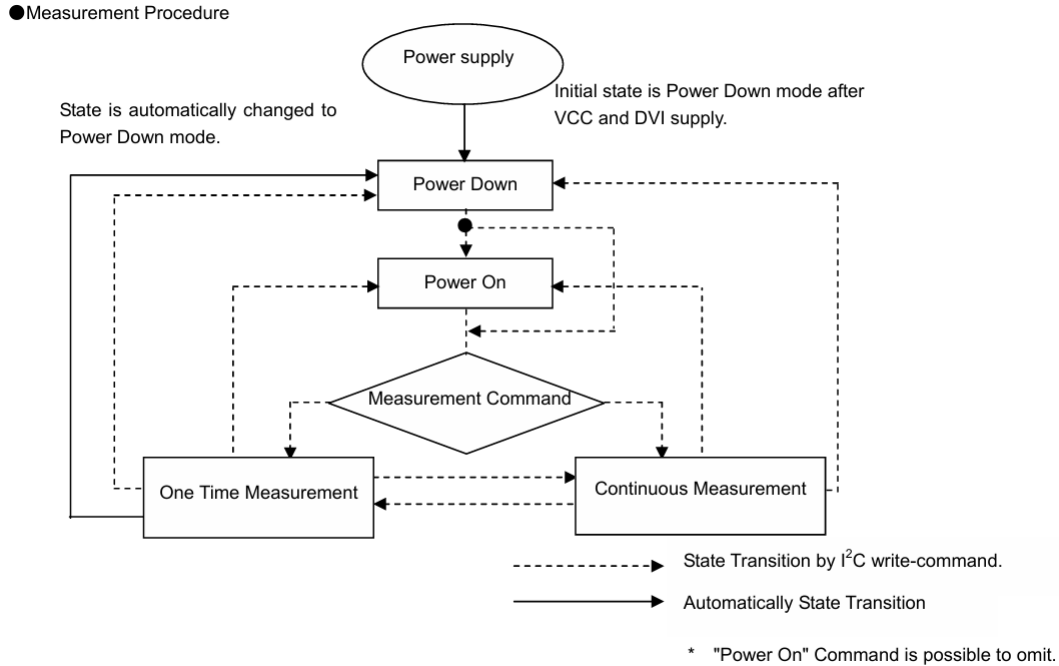


Figure 11: Lưu đồ Measurement Procedure của cảm biến BH1750

Quá trình thiết kế IP BH1750 được xây dựng dựa trên lưu đồ Measurement Procedure do nhà sản xuất cung cấp trong datasheet. Lưu đồ này mô tả toàn bộ trình tự hoạt động của cảm biến, bao gồm các trạng thái Power Down, Power On, gửi lệnh đo, chờ thời gian chuyển đổi và đọc dữ liệu. Trên cơ sở đó, máy trạng thái hữu hạn FSM của IP được xây dựng nhằm ánh xạ trực tiếp các trạng thái trong lưu đồ sang các trạng thái điều khiển tương ứng trong phần cứng.

Sau khi hệ thống được cấp nguồn và reset, IP BH1750 chuyển sang trạng thái khởi động POWER_UP và chờ cảm biến ổn định. Trong giai đoạn này, cảm biến ở chế độ Power Down theo mặc định. Khi kết thúc thời gian khởi tạo, IP bắt đầu quá trình giao tiếp I²C bằng cách tạo điều kiện Start và gửi địa chỉ ghi của cảm biến, sau đó truyền lệnh cấu hình chế độ đo Continuous High Resolution Mode. Chế độ này cho phép cảm biến thực hiện phép đo liên tục với độ phân giải cao, phù hợp với các ứng dụng giám sát ánh sáng theo chu kỳ.

Sau khi lệnh đo được gửi đi, IP chuyển sang trạng thái WAIT_MEASURE để chờ cảm biến hoàn tất quá trình chuyển đổi nội bộ. Khoảng thời gian chờ được tạo ra bằng bộ đếm nội bộ đồng bộ với xung clock hệ thống và được lựa chọn tương ứng với thời gian chuyển đổi tối đa được nêu trong datasheet của BH1750. Cách thiết kế này đảm bảo dữ liệu chỉ được đọc khi kết quả đo đã sẵn sàng, từ đó tránh sai lệch do đọc sớm.

Khi thời gian đo kết thúc, IP tạo lại điều kiện Start và gửi địa chỉ đọc của cảm biến. Tiếp theo, IP thực hiện đọc hai byte dữ liệu liên tiếp bao gồm byte cao và byte thấp của giá trị đo ánh sáng theo đúng thứ tự được quy định trong chuẩn giao tiếp I²C. Sau khi hoàn tất quá trình đọc, IP gửi tín hiệu Stop để kết thúc chu trình truyền thông. Giá trị ánh sáng thu được được ghép từ hai byte dữ liệu và xuất ra cổng *lux_data*, đồng thời tín hiệu *data_valid* được kích hoạt để báo hiệu dữ liệu hợp lệ.

Toàn bộ quá trình đo và đọc dữ liệu được lặp lại liên tục theo chu kỳ nhờ sự phối hợp với

IP Timer có sẵn trong hệ thống. Việc sử dụng IP Timer cho phép hệ thống linh hoạt thay đổi chu kỳ đo mà không cần chỉnh sửa cấu trúc IP BH1750, qua đó đáp ứng yêu cầu thiết kế hệ thống đo giá trị cảm biến theo chu kỳ với độ ổn định và độ chính xác cao.

Cảm biến BH1750FVI là một vi mạch tích hợp đo cường độ ánh sáng kỹ thuật số sử dụng giao thức truyền thông I²C. Trong thiết kế này, thực thể VHDL mang tên *BH1750_Driver* được xây dựng nhằm thực hiện việc giao tiếp, cấu hình và thu thập dữ liệu ánh sáng một cách tự động. Cấu trúc nội tại của cảm biến bao gồm diode quang, mạch chuyển đổi dòng điện sang điện áp và bộ chuyển đổi tương tự sang số ADC 16 bit để tạo ra dữ liệu số ở đầu ra.

Địa chỉ của cảm biến trên bus I²C được xác định bởi trạng thái của chân ADDR. Trong mã nguồn VHDL, hai hằng số *BH1750_ADDR_W* và *BH1750_ADDR_R* được định nghĩa tương ứng với trường hợp chân ADDR được nối đất, phù hợp với cấu hình phần cứng của hệ thống.

Trạng thái chân ADDR	Địa chỉ Slave 7 bit	Giá trị Write Read 8 bit
ADDR nối GND	0100011	0x46 / 0x47
ADDR nối VCC	1011100	0xB8 / 0xB9

Table 9: Thiết lập địa chỉ I²C cho BH1750

Hệ thống giao tiếp I²C được điều khiển bởi một máy trạng thái hữu hạn FSM nhằm đảm bảo trình tự truyền nhận dữ liệu tuân thủ đúng chuẩn giao thức. Trạng thái POWER_UP thực hiện việc chờ khoảng 10ms sau khi cấp nguồn để cảm biến ổn định. Sau đó, mã lệnh *CMD_H_RES_MODE* với giá trị x"10" được gửi để cấu hình chế độ đo liên tục độ phân giải cao với độ phân giải 1 lx.

Chế độ đo	Opcode	Độ phân giải
H-Resolution Mode	0001_0000	1 lx
H-Resolution Mode2	0001_0001	0.5 lx
L-Resolution Mode	0001_0011	4 lx

Table 10: Các chế độ đo chính của BH1750

Một điểm quan trọng trong thiết kế logic VHDL là việc sử dụng tín hiệu *i2c_tick* có tần số cao gấp bốn lần tần số SCL dự kiến. Cách tiếp cận này cho phép chia mỗi chu kỳ truyền bit thành bốn phân đoạn thời gian, qua đó đảm bảo các yêu cầu về thời gian thiết lập và giữ của tín hiệu SDA theo chuẩn I²C. Dữ liệu được lấy mẫu khi SCL ở mức cao, trong khi giá trị trên SDA chỉ được thay đổi khi SCL ở mức thấp.

ex1) Continuously H-resolution mode (ADDR = 'L')



① Send "Continuously H-resolution mode " instruction



② Wait to complete 1st H-resolution mode measurement.(max. 180ms.)

③ Read measurement result.

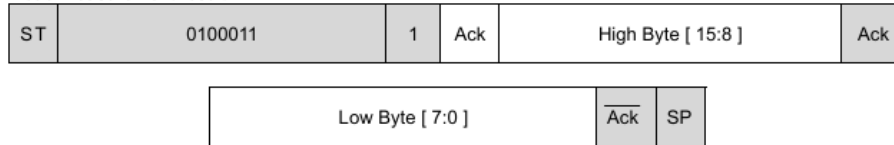


Figure 12: Trình tự thời gian đo trong chế độ Continuous High Resolution

Dữ liệu ánh sáng được đọc từ cảm biến bao gồm hai byte liên tiếp là High Byte và Low Byte. Sau khi hoàn tất quá trình đọc và kết thúc chu kỳ truyền thông, tín hiệu *data_valid* được đưa lên mức cao để báo hiệu dữ liệu mới đã sẵn sàng. Giá trị cường độ ánh sáng thực tế được tính toán bằng cách ghép hai byte dữ liệu và chia cho hệ số chuyển đổi mặc định của cảm biến.

$$Light_Intensity(lx) = \frac{High_Byte \times 2^8 + Low_Byte}{1.2} \quad (1)$$

Việc triển khai giao tiếp I²C được thực hiện theo phương pháp bit banging thông qua các bộ đệm ba trạng thái trên hai đường SDA và SCL, phù hợp với đặc tính hở cực thu của bus I²C. Tham số *DIVIDER* cho phép điều chỉnh linh hoạt tần số bus I²C dựa trên xung nhịp hệ thống, đồng thời đảm bảo tần số SCL không vượt quá giới hạn cho phép của cảm biến.

Thuật toán chương trình được triển khai như dưới đây:

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 entity BH1750_Driver is
6     Port (
7         clk          : in std_logic;
8         reset_n       : in std_logic;
9         sda           : inout std_logic;
10        scl           : inout std_logic;
11        lux_data       : out std_logic_vector(15 downto 0);
12        data_valid     : out std_logic
13    );
14 end BH1750_Driver;
15

```



```
16 architecture Behavioral of BH1750_Driver is
17
18     -- Clock divider for I2C
19     constant CLK_FREQ : integer := 100_000_000;
20     constant I2C_FREQ : integer := 100_000;
21     constant DIVIDER   : integer := CLK_FREQ / I2C_FREQ;
22
23     -- BH1750 I2C addresses
24     constant BH1750_ADDR_W : std_logic_vector(7 downto 0) := x"46";
25     constant BH1750_ADDR_R : std_logic_vector(7 downto 0) := x"47";
26     constant CMD_H_RES_MODE: std_logic_vector(7 downto 0) := x"10";
27
28     -- FSM states
29     type state_type is (
30         POWER_UP,
31         IDLE,
32         START_1,
33         SEND_ADDR_W,
34         ACK_1,
35         SEND_CMD,
36         ACK_2,
37         STOP_1,
38         WAIT_MEASURE,
39         START_2,
40         SEND_ADDR_R,
41         ACK_3,
42         READ_MSB,
43         M_ACK_1,
44         READ_LSB,
45         M_NACK,
46         STOP_2
47     );
48     signal state : state_type := POWER_UP;
49
50     signal clk_cnt : integer range 0 to DIVIDER := 0;
51     signal i2c_tick : std_logic := '0';
52
53     -- Delay counters
54     signal wait_timer : integer := 0;
55     constant WAIT_180MS : integer := 18_000_000;
56     constant WAIT_10MS  : integer := 1_000_000;
57
58     -- Data buffers
59     signal bit_cnt : integer range 0 to 7 := 0;
60     signal shift_reg : std_logic_vector(7 downto 0) := (others =>
61         '0');
62     signal data_msb : std_logic_vector(7 downto 0) := (others =>
63         '0');
```

```
62     signal data_lsb    : std_logic_vector(7 downto 0) := (others =>
        '0');
63
64     -- I2C signals
65     signal sda_out      : std_logic := '1';
66     signal sda_in_reg   : std_logic := '1';
67     signal scl_out      : std_logic := '1';
68
69 begin
70
71     -- Tri-state buffers
72     sda <= '0' when sda_out = '0' else 'Z';
73     scl <= '0' when scl_out = '0' else 'Z';
74
75     sda_in_reg <= sda;
76
77     -- I2C clock tick generator
78     process(clk)
79     begin
80         if rising_edge(clk) then
81             if reset_n = '0' then
82                 clk_cnt    <= 0;
83                 i2c_tick <= '0';
84             else
85                 if clk_cnt = (DIVIDER/4) - 1 then
86                     clk_cnt    <= 0;
87                     i2c_tick <= '1';
88                 else
89                     clk_cnt    <= clk_cnt + 1;
90                     i2c_tick <= '0';
91                 end if;
92             end if;
93         end if;
94     end process;
95
96     -- Main FSM
97     process(clk)
98         variable sub_state : integer range 0 to 3 := 0;
99     begin
100         if rising_edge(clk) then
101             if reset_n = '0' then
102                 state      <= POWER_UP;
103                 wait_timer <= 0;
104                 scl_out    <= '1';
105                 sda_out    <= '1';
106                 sub_state  := 0;
107                 lux_data   <= (others => '0');
108                 data_valid <= '0';
```

```
109     else
110
111         if state = POWER_UP or state = WAIT_MEASURE then
112             wait_timer <= wait_timer + 1;
113         else
114             wait_timer <= 0;
115         end if;
116
117         if i2c_tick = '1' then
118             case state is
119
120                 when POWER_UP =>
121                     if wait_timer > WAIT_10MS then
122                         state <= IDLE;
123                     end if;
124
125                 when IDLE =>
126                     sda_out <= '1';
127                     scl_out <= '1';
128                     state <= START_1;
129                     sub_state := 0;
130
131                 when START_1 =>
132                     case sub_state is
133                         when 0 => sda_out <= '1'; scl_out <=
134                             '1';
135                         when 1 => sda_out <= '0';
136                         when 2 => scl_out <= '0';
137                         when 3 =>
138                             state <= SEND_ADDR_W;
139                             bit_cnt <= 7;
140                             shift_reg <= BH1750_ADDR_W;
141                             sub_state := 0;
142                         when others => null;
143                     end case;
144                     if sub_state < 3 then sub_state :=
145                         sub_state + 1; end if;
146
147                 when SEND_ADDR_W | SEND_CMD | SEND_ADDR_R =>
148                     case sub_state is
149                         when 0 => sda_out <= shift_reg(
150                             bit_cnt); scl_out <= '0';
151                         when 1 => scl_out <= '1';
152                         when 2 => scl_out <= '1';
153                         when 3 =>
154                             scl_out <= '0';
155                             if bit_cnt = 0 then
156                                 if state = SEND_ADDR_W then
```

```
154         state <= ACK_1;
155         elsif state = SEND_CMD then
156             state <= ACK_2;
157         else state <= ACK_3;
158         end if;
159     else
160         bit_cnt <= bit_cnt - 1;
161     end if;
162 end case;
163 if sub_state < 3 then sub_state :=
164     sub_state + 1; else sub_state := 0;
165 end if;
166
167 when ACK_1 | ACK_2 | ACK_3 =>
168     case sub_state is
169         when 0 => sda_out <= '1'; scl_out <=
170             '0';
171         when 1 => scl_out <= '1';
172         when 2 => scl_out <= '1';
173         when 3 =>
174             scl_out <= '0';
175             if state = ACK_1 then
176                 state <= SEND_CMD;
177                 shift_reg <= CMD_H_RES_MODE;
178                 bit_cnt <= 7;
179             elsif state = ACK_2 then
180                 state <= STOP_1;
181             else
182                 state <= READ_MSB;
183                 bit_cnt <= 7;
184             end if;
185         end case;
186         if sub_state < 3 then sub_state :=
187             sub_state + 1; else sub_state := 0;
188         end if;
189
190 when STOP_1 =>
191     case sub_state is
192         when 0 => sda_out <= '0'; scl_out <=
193             '0';
194         when 1 => scl_out <= '1';
195         when 2 => sda_out <= '1';
196         when 3 => state <= WAIT_MEASURE;
197     end case;
198     if sub_state < 3 then sub_state :=
199         sub_state + 1; else sub_state := 0;
200     end if;
```

```
192         when WAIT_MEASURE =>
193             if wait_timer > WAIT_180MS then
194                 state <= START_2;
195                 sub_state := 0;
196             end if;
197
198         when START_2 =>
199             case sub_state is
200                 when 0 => sda_out <= '1'; scl_out <=
201                     '1';
202                 when 1 => sda_out <= '0';
203                 when 2 => scl_out <= '0';
204                 when 3 =>
205                     state <= SEND_ADDR_R;
206                     bit_cnt <= 7;
207                     shift_reg <= BH1750_ADDR_R;
208             end case;
209             if sub_state < 3 then sub_state :=
210                 sub_state + 1; else sub_state := 0;
211             end if;
212
213         when READ_MSB | READ_LSB =>
214             case sub_state is
215                 when 0 => sda_out <= '1'; scl_out <=
216                     '0';
217                 when 1 => scl_out <= '1';
218                 when 2 =>
219                     if state = READ_MSB then
220                         data_msb(bit_cnt) <=
221                             sda_in_reg;
222                     else
223                         data_lsb(bit_cnt) <=
224                             sda_in_reg;
225                     end if;
226                 when 3 =>
227                     scl_out <= '0';
228                     if bit_cnt = 0 then
229                         if state = READ_MSB then
230                             state <= M_ACK_1;
231                         else state <= M_NACK;
232                         end if;
233                     else
234                         bit_cnt <= bit_cnt - 1;
235                     end if;
236             end case;
237             if sub_state < 3 then sub_state :=
238                 sub_state + 1; else sub_state := 0;
239             end if;
```

```
231
232         when M_ACK_1 =>
233             case sub_state is
234                 when 0 => sda_out <= '0'; scl_out <=
235                     '0';
236                 when 1 => scl_out <= '1';
237                 when 2 => scl_out <= '1';
238                 when 3 =>
239                     scl_out <= '0';
240                     state <= READ_LSB;
241                     bit_cnt <= 7;
242             end case;
243             if sub_state < 3 then sub_state :=
244                 sub_state + 1; else sub_state := 0;
245             end if;
246
247         when M_NACK =>
248             case sub_state is
249                 when 0 => sda_out <= '1'; scl_out <=
250                     '0';
251                 when 1 => scl_out <= '1';
252                 when 2 => scl_out <= '1';
253                 when 3 =>
254                     scl_out <= '0';
255                     state <= STOP_2;
256             end case;
257             if sub_state < 3 then sub_state :=
258                 sub_state + 1; else sub_state := 0;
259             end if;
260
261         when STOP_2 =>
262             case sub_state is
263                 when 0 => sda_out <= '0'; scl_out <=
264                     '0';
265                 when 1 => scl_out <= '1';
266                 when 2 =>
267                     sda_out <= '1';
268                     lux_data <= data_msb & data_lsb;
269                     data_valid <= '1';
270                 when 3 => state <= WAIT_MEASURE;
271             end case;
272             if sub_state < 3 then sub_state :=
273                 sub_state + 1; else sub_state := 0;
274             end if;
275
276         when others =>
277             state <= IDLE;
```

```
270             end case;
271         end if;
272     end if;
273 end if;
274 end process;
275
276 end Behavioral;
```

Listing 1: BH1750_Driver.vhd

2.2.2 Thiết kế IP DHT11

Cảm biến DHT11 là cảm biến nhiệt độ và độ ẩm sử dụng giao thức truyền thông nối tiếp một dây (Single-Wire Two-Way Serial Interface). Trong thiết kế này, DHT11 được tích hợp dưới dạng một IP phần cứng nhằm thực hiện việc thu thập dữ liệu nhiệt độ và độ ẩm một cách tự động theo chu kỳ, độc lập với phần mềm điều khiển.

Giao tiếp giữa IP DHT11 và cảm biến được thực hiện thông qua một đường dữ liệu duy nhất, vừa truyền vừa nhận dữ liệu. Toàn bộ quá trình giao tiếp diễn ra trong khoảng 4 ms cho mỗi chu kỳ truyền, đảm bảo phù hợp với các ứng dụng đo lường theo chu kỳ trên nền tảng FPGA.

Định dạng dữ liệu truyền

Dữ liệu do cảm biến DHT11 gửi về có tổng độ dài 40 bit, trong đó mỗi byte được truyền theo thứ tự từ bit có trọng số cao đến bit có trọng số thấp. Cấu trúc gói dữ liệu bao gồm:

- 8 bit phần nguyên của độ ẩm (RH integer)
- 8 bit phần thập phân của độ ẩm (RH decimal)
- 8 bit phần nguyên của nhiệt độ (Temperature integer)
- 8 bit phần thập phân của nhiệt độ (Temperature decimal)
- 8 bit kiểm tra (Checksum)

Byte kiểm tra được tính bằng tổng của bốn byte dữ liệu đầu tiên. Nếu giá trị checksum nhận được trùng khớp với kết quả tính toán, dữ liệu được xem là hợp lệ và được IP chấp nhận để đưa ra hệ thống xử lý trung tâm.

Quy trình giao tiếp giữa IP DHT11 và cảm biến được mô tả như trong Hình 13. Ban đầu, cảm biến ở chế độ tiêu thụ năng lượng thấp. Khi IP DHT11 phát tín hiệu khởi động, cảm biến chuyển sang chế độ hoạt động và chuẩn bị gửi dữ liệu.

Sau khi hoàn tất quá trình truyền 40 bit dữ liệu, cảm biến quay trở lại chế độ tiết kiệm năng lượng và chỉ phản hồi khi nhận được tín hiệu khởi động mới từ IP.

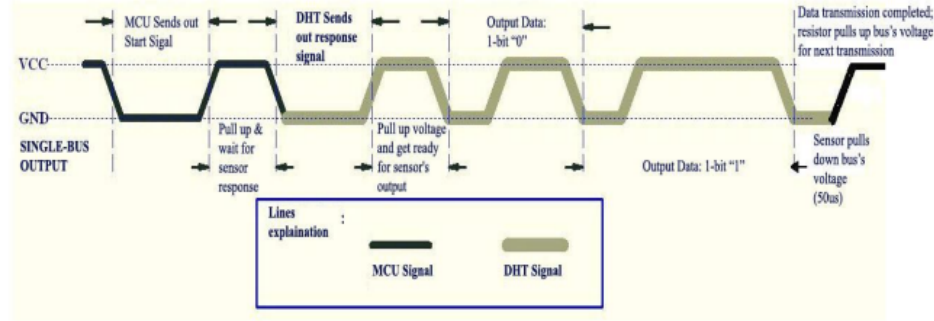


Figure 13: Quy trình giao tiếp tổng thể giữa IP DHT11 và cảm biến

Để bắt đầu một chu kỳ truyền dữ liệu, IP DHT11 chủ động kéo đường dữ liệu từ mức cao xuống mức thấp và giữ trong thời gian tối thiểu 18 ms nhằm đảm bảo cảm biến nhận diện được tín hiệu khởi động. Sau đó, IP nhả đường dữ liệu về mức cao và chờ trong khoảng 20–40 μs để đợi phản hồi từ cảm biến.

Quá trình này được minh họa trong Hình 14.

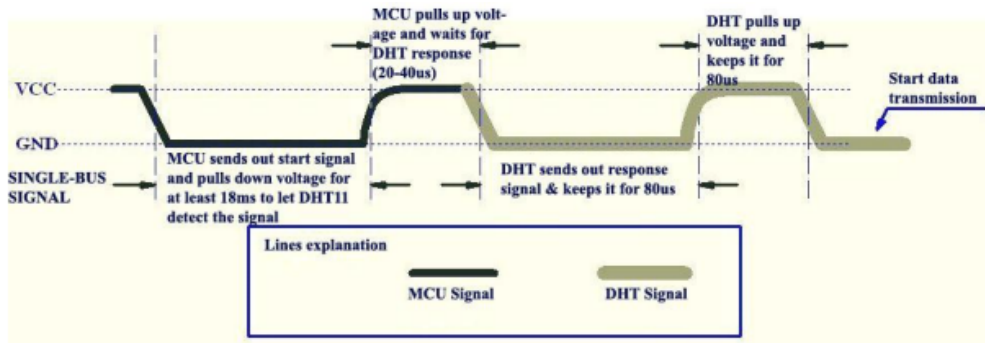


Figure 14: IP gửi tín hiệu khởi động và phản hồi từ DHT11

Sau khi phát hiện tín hiệu khởi động hợp lệ, DHT11 gửi tín hiệu phản hồi bằng cách kéo đường dữ liệu xuống mức thấp trong khoảng 80 μs , sau đó đưa lên mức cao thêm 80 μs để chuẩn bị truyền dữ liệu.

Mỗi bit dữ liệu được truyền theo cơ chế xung độ rộng, trong đó:

- Mỗi bit đều bắt đầu bằng mức thấp trong khoảng 50 μs .
- Thời gian mức cao phía sau quyết định giá trị bit:
 - Bit “0”: mức cao ngắn (khoảng 26–28 μs).
 - Bit “1”: mức cao dài (khoảng 70 μs).

Hai trường hợp biểu diễn bit “0” và bit “1” được thể hiện tương ứng trong Hình 15 và Hình 16.

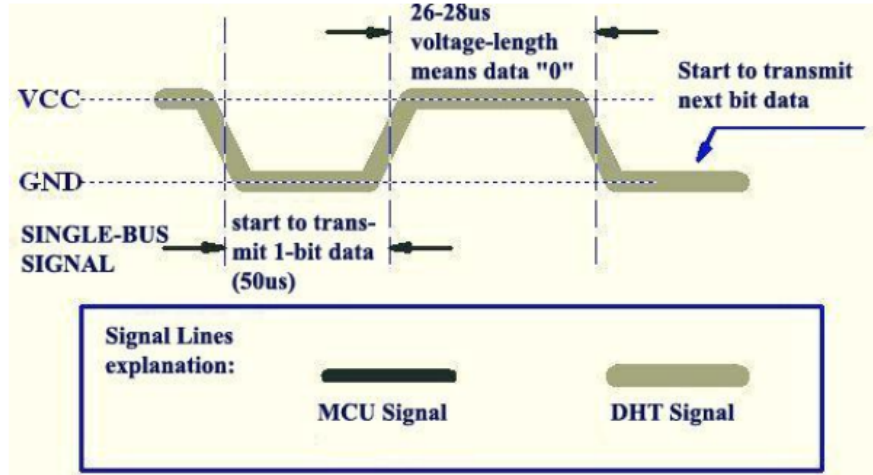


Figure 15: Biểu diễn truyền dữ liệu bit “0” của DHT11

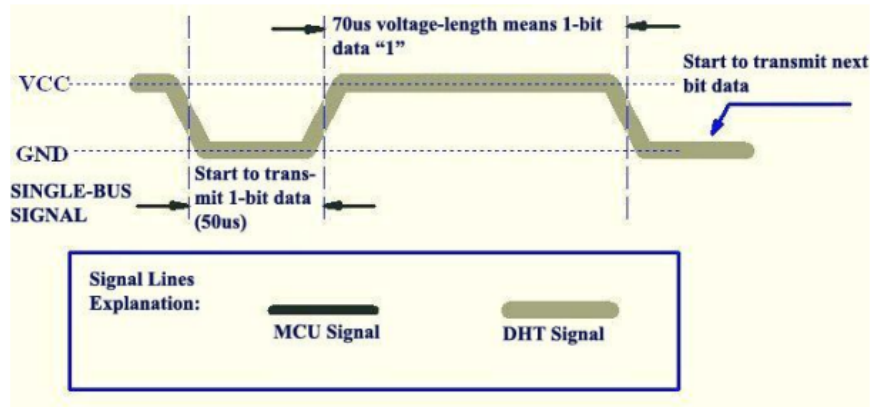


Figure 16: Biểu diễn truyền dữ liệu bit “1” của DHT11

Sau khi bit cuối cùng được truyền, DHT11 kéo đường dữ liệu xuống mức thấp trong khoảng $50 \mu s$, sau đó nhả bus để điện trở kéo lên đưa đường dữ liệu trở về trạng thái rảnh ở mức cao.

Trong thiết kế IP DHT11, toàn bộ quá trình trên được hiện thực bằng một máy trạng thái hữu hạn (FSM) đồng bộ với xung clock hệ thống. FSM chịu trách nhiệm tạo tín hiệu khởi động, đo chính xác độ rộng xung của từng bit dữ liệu và kiểm tra checksum trước khi xuất dữ liệu nhiệt độ và độ ẩm ra các thanh ghi nội bộ.

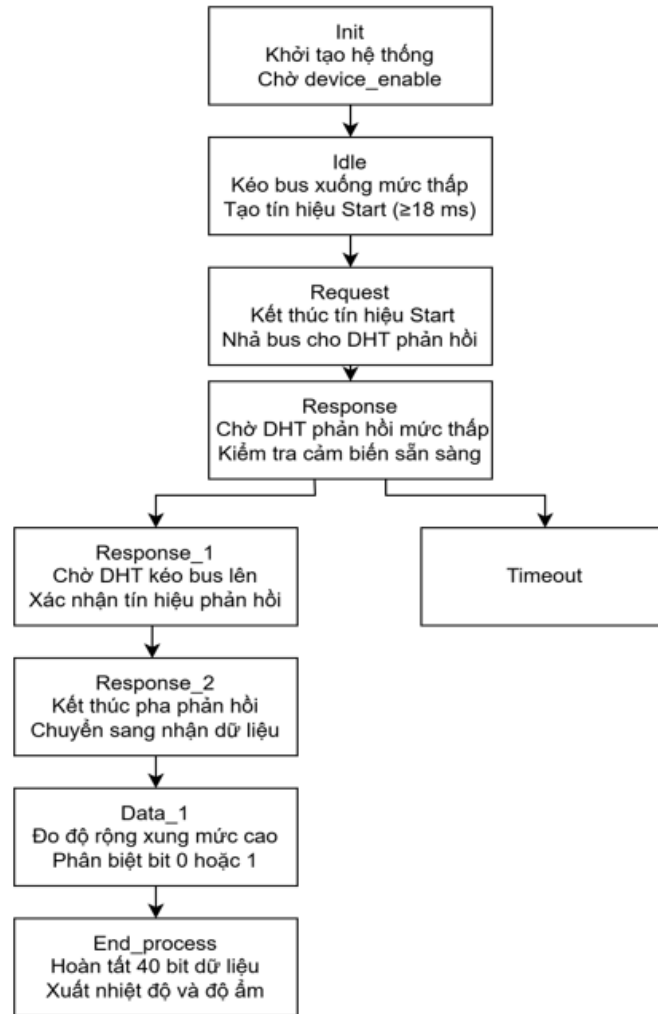


Figure 17: Sơ đồ máy trạng thái hữu hạn (FSM) điều khiển IP DHT11

Cách tiếp cận này cho phép IP DHT11 hoạt động hoàn toàn trên phần cứng, đảm bảo tính ổn định, độ chính xác về thời gian và phù hợp với kiến trúc hệ thống đọc cảm biến theo chu kỳ trên FPGA Arty A7-35T.

Chương trình đầy đủ được triển khai dưới đây:

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 library UNISIM;
6 use UNISIM.VComponents.all;
7
8 entity DHT11_Driver is
9 Port (
10 data_in : inout STD_LOGIC;
11 clk : in STD_LOGIC;

```

```
12 device_enable : in std_logic;
13 temp : out std_logic_vector(7 downto 0);
14 end_p : out std_logic;
15 humidity : out std_logic_vector(7 downto 0);
16 debug : out std_logic_vector(31 downto 0);
17 crc : out std_logic_vector(7 downto 0);
18 rst : in std_logic
19 );
20 end DHT11_Driver;
21
22 architecture Behavioral of DHT11_Driver is
23
24 -- Constants for 100MHz Clock (Based on original calculation)
25 constant Delay_18_ms : positive := 18 * 10**6 / 10 + 1;
26 constant Delay_30_us : positive := 30 * 10**3 / 10 + 1;
27 constant Delay_50_us : positive := 50 * 10**3 / 10 + 1;
28 constant Delay_75_us : positive := 75 * 10**3 / 10 + 1;
29 constant Delay_80_us : positive := 80 * 10**3 / 10 + 1;
30 constant Delay_100_ms : positive := 100 * 10**6 / 10 + 1;
31 constant Delay_1s : positive := 1000 * 10**6 / 10 + 1;
32
33 signal input_sync : std_logic_vector(1 downto 0);
34 type state is (Init, Idle, Request, Response, Response_1, Response_2
35 , Data, Data_1, End_process);
36 signal rising_sig, falling_sig : boolean;
37 signal curr_state, next_state : state;
38 signal rst_counter : std_logic;
39 signal count : integer;
40 signal en : std_logic;
41 signal data_s : std_logic;
42 signal data_count : integer range 0 to 40;
43 signal data_word : std_logic_vector(39 downto 0);
44 signal dec : std_logic;
45 signal set_one, set_zero : std_logic;
46
47 begin
48 -- Signal edge detector
49 edge_detect: process(clk, rst)
50 begin
51 if rising_edge(clk) then
52 if rst = '0' then
53 input_sync <= "00";
54 else
55 input_sync <= input_sync(0) & data_s;
56 end if;
57 end if;
58 end process;
```

```
59
60 rising_sig <= input_sync(1 downto 0) = "01";
61 falling_sig <= input_sync(1 downto 0) = "10";
62
63 counter: process(clk)
64 begin
65 if rising_edge(clk) then
66 if rst = '0' then
67 count <= 0;
68 data_word <= (others => '0');
69 else
70 if rst_counter = '1' then
71 count <= 0;
72 else
73 count <= count + 1;
74 end if;
75 end if;
76
77 if dec = '1' then
78 data_count <= data_count - 1;
79 end if;
80
81 if curr_state = Request then
82 data_count <= 40;
83 data_word <= (others => '0');
84 end if;
85
86 if set_one = '1' then
87 data_word(data_count - 1) <= '1';
88 data_count <= data_count - 1;
89 end if;
90
91 if set_zero = '1' then
92 data_word(data_count - 1) <= '0';
93 data_count <= data_count - 1;
94 end if;
95
96 if curr_state = End_process then
97 temp <= data_word((40 - (8 * 2)) - 1 downto (40 - (8 * 3)));
98 humidity <= data_word((40 - (8 * 0)) - 1 downto (40 - (8 * 1)));
99 debug <= data_word(39 downto 8);
100 crc <= data_word(7 downto 0);
101 end if;
102 end if;
103 end process;
104
105 clk_process: process(clk, rst)
106 begin
```

```
107 if rising_edge(clk) then
108 if rst = '0' then
109 curr_state <= Init;
110 else
111 curr_state <= next_state;
112 end if;
113 end if;
114 end process;
115
116 fsm: process(curr_state, count, falling_sig, rising_sig, data_count,
117             device_enable)
118 begin
119 next_state <= curr_state;
120 rst_counter <= '0';
121 en <= '1';
122 dec <= '0';
123 set_one <= '0';
124 set_zero <= '0';
125 end_p <= '0';
126
127 case curr_state is
128 when Init =>
129 if device_enable = '1' then
130 rst_counter <= '1';
131 next_state <= Idle;
132 end if;
133
134 when Idle =>
135 en <= '0';
136 if count = Delay_18_ms then
137 next_state <= Request;
138 rst_counter <= '1';
139 end if;
140
141 when Request =>
142 next_state <= Response;
143
144 when Response =>
145 if falling_sig then
146 next_state <= Response_1;
147 rst_counter <= '1';
148 end if;
149 if count > Delay_100_ms then
150 next_state <= Init;
151 end if;
152
153 when Response_1 =>
154 if rising_sig then
```

```
154 next_state <= Response_2;
155 rst_counter <= '1';
156 end if;
157
158 if count > Delay_100_ms then
159 next_state <= Init;
160 end if;
161
162 when Response_2 =>
163 if falling_sig then
164 next_state <= Data;
165 rst_counter <= '1';
166 end if;
167
168 if count > Delay_100_ms then
169 next_state <= Init;
170 end if;
171
172 when Data =>
173 if rising_sig then
174 next_state <= Data_1;
175 rst_counter <= '1';
176 end if;
177
178 if data_count = 0 then
179 rst_counter <= '1';
180 next_state <= End_process;
181 end if;
182
183 if count > Delay_100_ms then
184 next_state <= Init;
185 end if;
186
187 when Data_1 =>
188 if falling_sig then
189 if count <= Delay_30_us then
190 set_zero <= '1';
191 elsif count <= Delay_75_us then
192 set_one <= '1';
193 else
194 -- invalid pulse width
195 end if;
196 next_state <= Data;
197 end if;
198
199 if count > Delay_100_ms then
200 next_state <= Init;
201 end if;
```

```
202
203 when End_process =>
204 if count = Delay_1s then
205 next_state <= Init;
206 rst_counter <= '1';
207 end_p <= '1';
208 end if;
209
210 when others =>
211 next_state <= Init;
212
213 end case;
214
215 end process;
216
217 -- IO buffer
218 u1: iobuf
219 generic map(
220 drive => 12,
221 iostandard => "lvcmos33",
222 slew => "slow")
223 port map(
224 o => data_s,
225 io => data_in,
226 i => '0',
227 t => en
228 );
229
230 end Behavioral;
```

Listing 2: DHT11_Driver.vhd

CHƯƠNG 3: KẾT QUẢ VÀ ĐÁNH GIÁ

3.1 Xây dựng hệ thống trên Vivado

Ta xây dựng khối *Block Design* của hệ thống trên Vivado như minh họa trong Hình 18. Trong đó, bộ xử lý MicroBlaze giao tiếp với các IP ngoại vi thông qua bus AXI4-Lite, bao gồm AXI Timer, IP cảm biến DHT11 và IP cảm biến BH1750.

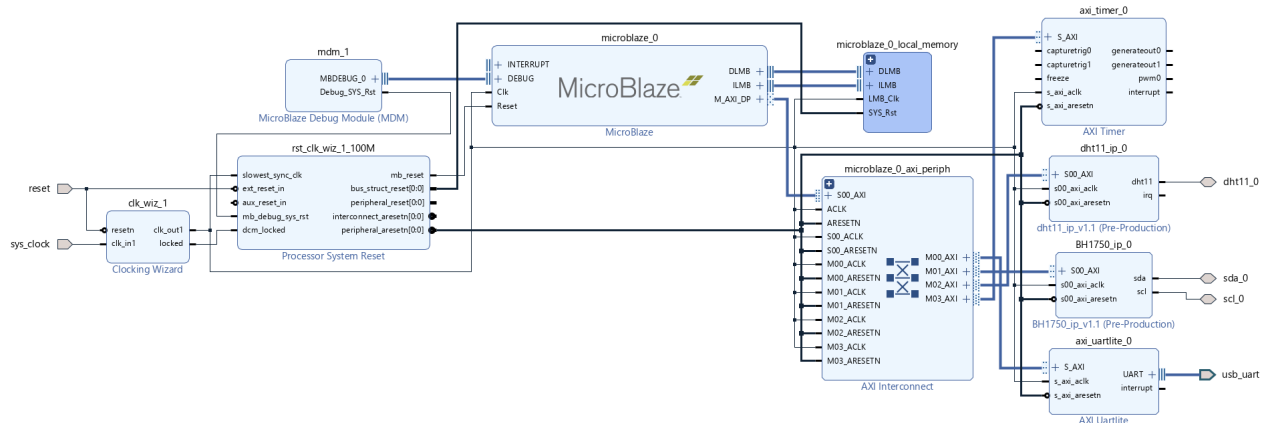


Figure 18: Sơ đồ khối Block Design của hệ thống

Các cổng vào/ra (I/O Ports) của hệ thống được cấu hình như thể hiện trong Hình 19, đóng vai trò kết nối giữa phần cứng trên FPGA và các tín hiệu bên ngoài.

Name	Direction	Board Part Pin	Board Part Interface	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type
▼ All ports (7)													
▼ ext_reset_58201 (1)	IN					✓	35	LVC MOS33*	3.300				NONE
▼ Scalar ports (1)													
reset	IN	reset			C2	✓	35	LVC MOS33*	3.300				NONE
▼ UART_28804 (2)	(Multiple)					✓	16	LVC MOS33*	3.300				NONE
▼ Scalar ports (2)													
usb_uart_rxd	IN	usb_uart_rxd			A9	✓	16	LVC MOS33*	3.300				NONE
usb_uart_txd	OUT	usb_uart_txd			D10	✓	16	LVC MOS33*	3.300	12			NONE
▼ Scalar ports (4)													
dht11_0	INOUT				G13	✓	15	LVC MOS33*	3.300	12		NONE	PULLUP
scl_0	INOUT				L18	✓	14	LVC MOS33*	3.300	12			PULLUP
sda_0	INOUT				M18	✓	14	LVC MOS33*	3.300	12			PULLUP
sys_clock	IN	clk			E3	✓	35	LVC MOS33*	3.300				NONE

Figure 19: Cấu hình các cổng I/O của hệ thống

Chương trình C được triển khai trên Vitis:

```

1
2 #include <stdio.h>
3 #include "platform.h"
4 #include "xil_printf.h"
5 #include "xparameters.h"
6 #include "sleep.h"
7 #include "xtmrctr.h"

```



```
8 #include "BH1750_ip.h"
9 #include "dht11_ip.h"
10
11
12 #define TMR_DEVICE_ID    XPAR_TMRCTR_0_DEVICE_ID
13 #define TIMER_0          0
14
15 #define RESET_VALUE      300000000
16
17
18 #define BH1750_BASE_ADDR  XPAR_BH1750_IP_0_S00_AXI_BASEADDR
19 #define DHT11_BASE_ADDR  XPAR_DHT11_IP_0_S00_AXI_BASEADDR
20
21
22 XTmrCtr tmr;
23
24 int main()
25 {
26     init_platform();
27     int status;
28
29
30     status = XTmrCtr_Initialize(&tmr, TMR_DEVICE_ID);
31     if (status != XST_SUCCESS) {
32         xil_printf("Timer_Init_Failed\r\n");
33         return XST_FAILURE;
34     }
35
36     XTmrCtr_Stop(&tmr, TIMER_0);
37     XTmrCtr_SetResetValue(&tmr, TIMER_0, RESET_VALUE);
38     XTmrCtr_SetOptions(&tmr, TIMER_0, XTC_AUTO_RELOAD_OPTION |
39         XTC_DOWN_COUNT_OPTION);
40     XTmrCtr_Start(&tmr, TIMER_0);
41
42     u32 bh_raw_data;
43     u32 lux_int, lux_dec, lux_x10;
44     u32 dht_temp = 0, dht_humid = 0;
45
46     print("=====\n\r");
47     print("TONG HOP CAM BIEN (USE LIB HEADER)\n\r");
48     print("=====\n\r");
49
50     while(1) {
51
52         if (XTmrCtr_IsExpired(&tmr, TIMER_0)) {
```

```
55     XTmrCtr_Stop(&tmr, TIMER_0);
56
57
58     DHT11_IP_mWriteReg(DHT11_BASE_ADDR,
59         DHT11_IP_S00_AXI_SLV_REG0_OFFSET, 0x01);
60
61     usleep(30000);
62
63
64     dht_temp = DHT11_IP_mReadReg(DHT11_BASE_ADDR,
65         DHT11_IP_S00_AXI_SLV_REG2_OFFSET);
66
67     dht_humid = DHT11_IP_mReadReg(DHT11_BASE_ADDR,
68         DHT11_IP_S00_AXI_SLV_REG3_OFFSET);
69
70     DHT11_IP_mWriteReg(DHT11_BASE_ADDR,
71         DHT11_IP_S00_AXI_SLV_REG0_OFFSET, 0x00);
72
73
74     bh_raw_data = BH1750_IP_mReadReg(BH1750_BASE_ADDR,
75         BH1750_IP_S00_AXI_SLV_REG0_OFFSET);
76
77     lux_x10 = (bh_raw_data * 100) / 12;
78     lux_int = lux_x10 / 10;
79     lux_dec = lux_x10 % 10;
80
81
82
83     xil_printf("-----\n\r");
84     xil_printf("Status: Doc\sau\3\giay\ (Timer)\n\r");
85     xil_printf("Do\am\ (DHT11):\%\lu\%\n\r", dht_humid);
86     xil_printf("Nhiet\do\ (DHT11):\%\lu\C\n\r", dht_temp);
87     xil_printf("Anh\sang\ (BH1750):\%d.%d\lx\n\r", lux_int,
88         lux_dec);
89
90     XTmrCtr_Reset(&tmr, TIMER_0);
91     XTmrCtr_Start(&tmr, TIMER_0);
92 }
93
94 cleanup_platform();
95 return 0;
96 }
```

Listing 3: Chương trình chính của hệ thống

Toàn bộ mã nguồn của dự án, bao gồm cấu hình phần cứng (Vivado) và phần mềm điều khiển (Vitis), video quay lại hoạt động của hệ thống đã được lưu trữ tại:

Mã nguồn dự án

<https://github.com/dathavan1523/fpga-ip-sensor-arty7>

3.2 Kết quả và đánh giá

Sau khi hoàn thiện thiết kế phần cứng trên Vivado và lập trình phần mềm trên Vitis, hệ thống được nạp và chạy thử nghiệm trên FPGA. AXI Timer được cấu hình tạo chu kỳ đọc dữ liệu 3 giây, tại mỗi chu kỳ hệ thống tiến hành thu thập dữ liệu từ hai cảm biến DHT11 và BH1750, sau đó hiển thị kết quả thông qua cổng UART.

Kết quả thu được trong quá trình thực nghiệm được minh họa trong Hình 20.

Connected to: Serial (COM9, 9600, 0, 8)

Thời gian thực: Đọc sau 3 giây

Do ẩm: 54 %

Nhiệt độ: 26 C

Ánh sáng: 208.3 lx (Raw: 250)

Thời gian thực: Đọc sau 3 giây

Do ẩm: 52 %

Nhiệt độ: 26 C

Ánh sáng: 210.0 lx (Raw: 252)

Thời gian thực: Đọc sau 3 giây

Do ẩm: 51 %

Nhiệt độ: 26 C

Ánh sáng: 210.8 lx (Raw: 253)

Thời gian thực: Đọc sau 3 giây

Do ẩm: 50 %

Nhiệt độ: 26 C

Ánh sáng: 210.0 lx (Raw: 252)

Figure 20: Kết quả đọc dữ liệu cảm biến hiển thị qua UART theo chu kỳ 3 giây

Từ Hình 20 có thể nhận thấy rằng hệ thống hoạt động ổn định và đúng yêu cầu thiết kế:

Thời gian đọc dữ liệu được thực hiện đúng theo chu kỳ 3 giây, chứng tỏ AXI Timer hoạt động chính xác.

Nhiệt độ đo được từ cảm biến DHT11 ổn định ở mức 26°C, cho thấy quá trình giao tiếp và xử lý dữ liệu không xảy ra lỗi.

Độ ẩm có sự thay đổi nhẹ qua các lần đo (từ 54 xuống 50), phù hợp với đặc tính biến đổi tự nhiên của môi trường.

Giá trị cường độ ánh sáng đo được từ cảm biến BH1750 dao động trong khoảng 208–211 lux, trong khi giá trị raw chỉ thay đổi nhỏ, cho thấy IP cảm biến hoạt động ổn định và việc chuyển đổi dữ liệu được thực hiện đúng.

Kết quả thực nghiệm cho thấy hệ thống đáp ứng tốt yêu cầu đề tài, đảm bảo khả năng đọc dữ liệu cảm biến theo chu kỳ định sẵn và hiển thị kết quả một cách liên tục.

CHƯƠNG 4: KẾT LUẬN

4.1 Ưu điểm

Thông qua quá trình thiết kế và triển khai hệ thống, đề tài đạt được một số ưu điểm nổi bật như sau.

Thứ nhất, đề tài giúp củng cố và mở rộng kiến thức về thiết kế IP Core trên nền tảng FPGA. Việc xây dựng các IP cảm biến DHT11 và BH1750 dưới dạng phần cứng cho phép người thực hiện nắm vững quy trình thiết kế IP chuẩn, từ mô tả chức năng, xây dựng máy trạng thái hữu hạn (FSM), cho đến đóng gói IP và tích hợp vào hệ thống thông qua giao tiếp AXI4-Lite.

Thứ hai, hệ thống đạt độ ổn định và chính xác cao nhờ tiếp cận theo hướng phần cứng hóa. Các giao thức truyền nhận của cảm biến được xử lý trực tiếp trong phần cứng, đảm bảo yêu cầu về thời gian và khắc phục hạn chế của các phương pháp định thời bằng phần mềm, đặc biệt đối với các giao thức nhạy cảm về timing như DHT11.

Thứ ba, kiến trúc hệ thống được xây dựng theo mô hình đồng thiết kế phần cứng/phần mềm (Hardware/Software Co-design). AXI Timer đảm nhiệm vai trò định thời chu kỳ lấy mẫu, trong khi bộ xử lý MicroBlaze chỉ thực hiện việc điều khiển và đọc dữ liệu thông qua các thanh ghi ánh xạ bộ nhớ. Cách tiếp cận này giúp giảm tải cho bộ xử lý và tăng khả năng mở rộng của hệ thống.

Cuối cùng, việc tuân thủ chuẩn giao tiếp AXI4-Lite giúp các IP Core trong đề tài có tính chuẩn hóa và khả năng tái sử dụng cao. Các IP có thể dễ dàng tích hợp vào các hệ thống FPGA khác hoặc mở rộng cho các ứng dụng phức tạp hơn mà không cần thay đổi cấu trúc nội bộ.

4.2 Nhược điểm

Mặc dù đạt được các mục tiêu đề ra, hệ thống vẫn còn tồn tại một số hạn chế nhất định.

Thứ nhất, việc sử dụng bộ xử lý phần mềm MicroBlaze kết hợp với nhiều IP Core riêng biệt dẫn đến mức tiêu thụ tài nguyên logic (LUTs, Flip-Flops) tương đối lớn so với các giải pháp vi điều khiển truyền thống. Đối với các ứng dụng đơn giản chỉ yêu cầu đọc dữ liệu cảm biến, giải pháp FPGA có thể chưa tối ưu về mặt chi phí và hiệu năng.

Thứ hai, giao diện người dùng của hệ thống còn đơn giản. Kết quả đo hiện tại chỉ được hiển thị dưới dạng dữ liệu văn bản thông qua cổng UART, chưa có giao diện trực quan tại chỗ hoặc khả năng giám sát từ xa. Điều này phần nào hạn chế khả năng ứng dụng thực tế của hệ thống trong các bài toán IoT.

Ngoài ra, khả năng xử lý lỗi của hệ thống chưa thực sự hoàn thiện. Mặc dù IP DHT11 đã có cơ chế kiểm tra checksum, hệ thống chưa được trang bị các chức năng xử lý lỗi nâng cao như tự động khởi động lại cảm biến khi treo hoặc cảnh báo khi giá trị đo vượt ngưỡng an toàn.

4.3 Hướng phát triển

Từ những kết quả đạt được và các hạn chế đã phân tích, đề tài có thể được tiếp tục phát triển theo một số hướng sau.

Trong tương lai, hệ thống có thể được nâng cấp bằng cách tích hợp cơ chế ngắt (interrupt) cho các IP Core và AXI Timer. Khi đó, bộ xử lý MicroBlaze chỉ được kích hoạt khi có sự kiện hoàn tất đo hoặc phát hiện lỗi, giúp tối ưu hơn nữa hiệu suất xử lý và tiêu thụ năng lượng.

Bên cạnh đó, hệ thống có thể mở rộng sang các bài toán điều khiển tự động vòng kín. Việc bổ sung các IP Core điều khiển cơ cấu chấp hành như PWM cho quạt làm mát hoặc relay cho hệ thống chiếu sáng, kết hợp với dữ liệu từ cảm biến, sẽ cho phép xây dựng các hệ thống tự động hóa hoàn chỉnh thay vì chỉ dừng lại ở mức giám sát.

Một hướng phát triển quan trọng khác là tích hợp khả năng kết nối mạng. Hệ thống có thể khai thác cổng Ethernet trên kit Arty A7-35T hoặc giao tiếp với module WiFi để truyền dữ liệu lên máy chủ hoặc nền tảng đám mây, từ đó phát triển thành một node IoT hoàn chỉnh.

Cuối cùng, việc tối ưu hóa tài nguyên và công suất tiêu thụ cũng là một hướng nghiên cứu tiềm năng. Thông qua việc phân tích báo cáo timing và power trong Vivado, mã nguồn RTL có thể được tối ưu để giảm diện tích logic, giúp hệ thống có thể triển khai trên các dòng FPGA chi phí thấp hơn.

TÀI LIỆU THAM KHẢO

- [1] D. Inc., *Arty A7 - Digilent Reference*. Digilent. FPGA Artix-7 XC7A35T Development Board.
- [2] ROHM Semiconductor, “Bh1750 digital light sensor datasheet.” Available at: <https://www.mouser.com/datasheet/2/348/BH1750FVI-E-1860643.pdf>, 2020. Accessed: 6 January 2026.
- [3] Various Manufacturers, “Dht11 temperature and humidity sensor datasheet.” Available at: <https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT11.pdf>, 2021. Accessed: 6 January 2026.
- [4] Advanced Micro Devices, Inc., “Axi timer: Functional description.” Available at: <https://docs.amd.com/r/en-US/pg079-axi-timer/Functional-Description>, 2024. Accessed: 6 January 2026.
- [5] ARM Limited, “Amba axi protocol specification v2.” Available at: IHI0022L_amba_axi_protocol_spec_2.pdf, 2010. Accessed: 6 January 2026.
- [6] Advanced Micro Devices, Inc., “Vivado design suite.” AMD Official Website. Available at: <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html>, 2024. Accessed: 15 December 2025.